

On Leveraging Pretrained Transformer Models for Automatic Speech Recognition

by

Aravind Krishnan

Matriculation Number : 7039228

Master Thesis

Submitted as part of the degree requirements of the

MSc in Language Science and Technology

Department of Language Science and Technology

Faculty of Philosophy

Saarland University

Supervisor

Prof. Dr. Dietrich Klakow

Advisor

Jesujoba Alabi

March 16, 2026



**UNIVERSITÄT
DES
SAARLANDES**

Declaration of Authorship

Einverständniserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Ich versichere, dass die gedruckte und die elektronische Version der Masterarbeit inhaltlich übereinstimmen.

Declaration of Consent

I hereby confirm that the thesis presented here is my own work, with all assistance acknowledged. I assure that the electronic version is identical in content to the printed version of the Master's thesis.

Ort/Place: Saarbrücken (Saarland)

Datum/Date:

Unterschrift/Signature:

On Leveraging Pretrained Transformer Models for Automatic Speech Recognition

by Aravind Krishnan

Matriculation Number : 7039228

Abstract

The introduction of the transformer architecture and the pretrain-finetune paradigm has revolutionized all areas of AI research. Natural Language Processing (NLP) research has arguably benefited most from this paradigm, but other fields like computer vision and speech processing have also rapidly adopted these techniques and methods. This thesis looks at the adoption of pretrained transformer models within the Automatic Speech Recognition (ASR) pipeline.

In the first part, we look at how pretrained text language models (PLMs) can be used as linguistic components in ASR models. Using domain-specific scenarios as targets, we justify and demonstrate their approximation into n-gram models that can be used during first pass decoding. We find that continued pretraining on domain-specific text, followed by large-scale random sampling can produce considerably approximate n-gram models that can then be strengthened by interpolation. We explore the challenges and factors that affect sampling-based n-gram approximation, and find that it is highly beneficial in few-shot scenarios. The second part of the thesis looks at pretrained speech transformer (SSL) models: we look at their representations to study how they organize and use speaker attributes (gender) and linguistic information. Our experiments with frame-level probing reveal a concentration of gender information in the first/last frame-representations across all layers, especially in the ASR models. Using linear erasure, we remove linearly encoded gender from SSL representations and find that it has very little effect on downstream ASR performance.

Acknowledgements

I would like to begin by expressing my gratitude to my supervisor, Prof. Dietrich Klakow, for his guidance, encouragement, and patience throughout my time in Saarbrücken. I've learned mountains of things from him these last few years and I'm grateful for everything he has done to shape my academic life. I am also grateful to the Language Science and Technology department, the administrative team (especially Claudia) and my fellow classmates for their support.

I'd also like to thank all my wonderful colleagues and friends at LSV for several years of support, friendship and advice. Deadlines, rejections and acceptances have come and gone and we've kept the ship floating and offered support to each other wherever we could. I look forward to more adventures, cake breaks and mensa lunches in their company.

Throughout my master studies, it has been pointed out to me (hundreds of times) that keeping me alive takes a village. As I've come to accept this truth, I'd like to thank the village chief – Julian – and my fellow villagers – Cynthia, Ken, and Pepe. They have gone through this arguably as much as I have and kept me happy, sane and grounded. I am also indebted to Maribel and Stefan for offering me a calming refuge filled with food, love and cake whenever life in Saarbrücken got too hectic.

I offer my thanks to Ubu Le Roi for always providing the coffee and ambiance I needed when writing assignments or papers. Additional thanks to cafe Luuc for providing all this plus charging ports (Ubu has one plug to charge your laptop my lap I never had charge on mine).

My parents and siblings have stood by me long before I met anyone else on this acknowledgment. I've made several unconventional choices for the surroundings I grew up in, and I've received nothing but unwavering support from each one of them all these years. Ambi, Govi, Acha and Amma, thank you very much for everything.

CONTENT

Declaration of Authorship	i
Abstract	ii
Acknowledgements	iii
1 Linguistic Transformers in Speech	1
1.1 Related Work	3
2 Approximating PLMs into n-grams	5
2.1 Data	5
2.2 Approximating GPT-2	5
2.2.1 Sampling-Based Approximation	6
2.2.2 Probability-Based Approximation	6
2.2.3 Vocabulary-Restricted Decoding	7
2.3 Experimental Setup	8
2.4 Comparing Probability and Sampling Approximation	8
2.5 Vocabulary Restriction and Interpolation	10
3 Factors Affecting Sampling based Approximation	12
3.0.1 Domain difficulty	12
3.0.2 Few-Shot Approximation	14
4 Acoustic Transformers in Speech	15
4.1 Gender and SSL models	18
5 Gender Erasure	20
5.0.1 Concept Scrubbing	20
5.1 Data	21
5.2 Experimental Setup	22
5.3 Gender Scrubbing	22

5.3.1	Tracking Erasure	23
5.3.2	Effects on downstream ASR	24
6	On Pooling and Probing	26
6.0.1	The Problem with Mean Pooling	26
6.0.2	Snapshot probing	28
6.0.3	Cross-Position Snapshot probing	29
7	Conclusions and future work	31
7.1	Future work	32
7.1.1	SSL models and feature erasure	32
7.1.2	Transformers and Speech Models	32
	Bibliography	37

CHAPTER 1

LINGUISTIC TRANSFORMERS IN SPEECH

Automatic Speech Recognition (ASR) is the technology that enables the conversion of spoken language into written text. It uses techniques from signal processing, machine learning, and natural language processing to transcribe an audio input. ASR is widely used in applications like virtual assistants, transcription services, and voice-controlled systems, making human-computer interaction more natural and accessible. [Figure 1.1](#) depicts an ASR module. It consists of two major components: The acoustic model and the language model. The acoustic model is responsible for converting speech signals into phonemes that eventually compose words. The language model imposes grammar and other linguistic rules on the word sequences produced by the acoustic model. Transcription works through the collaboration of both components, controlled by a third component called the decoder.

In the pre-neural, statistical ASR era (1990s–2010s), decoders were typically implemented as weighted finite-state transducers (WFSTs) [1–3]. The acoustic model was an HMM whose state emission probabilities were parameterized with Gaussian Mixture Models (GMMs), and modeled context-dependent phonetic units such as triphones. Language models were separately trained as n-gram models [4, 5] over sequences of text. WFST composition was then used to combine the acoustic model, lexicon, and language model into a single search space, forming a statistical speech recognizer. With advancements in deep learning, GMM-HMM based acoustic models were replaced with neural models that encoded speech. The development of the Connectionist Temporal Classification (CTC) [6] algorithm meant that a neural network could be trained to map audio frames into characters without pre-alignment. Several acoustic encoder architectures were explored as a result, like Deep Neural Networks (DNNs) [7], Convolutional Neural Networks (CNNs) [8, 9], and Recurrent Neural networks (RNNs)[10, 11]. Although these acoustic encoders could be readily used for transcription, the n-gram models were kept intact since these networks did not inherently model complex language given the context limits of these architectures and the modeling assumptions inherent to CTC (i.e., conditional independence). The introduction of the attention mechanism [12] and the transformer architecture [13] significantly increased the contextual capacity of neural models. This, along with the introduction of the pretraining

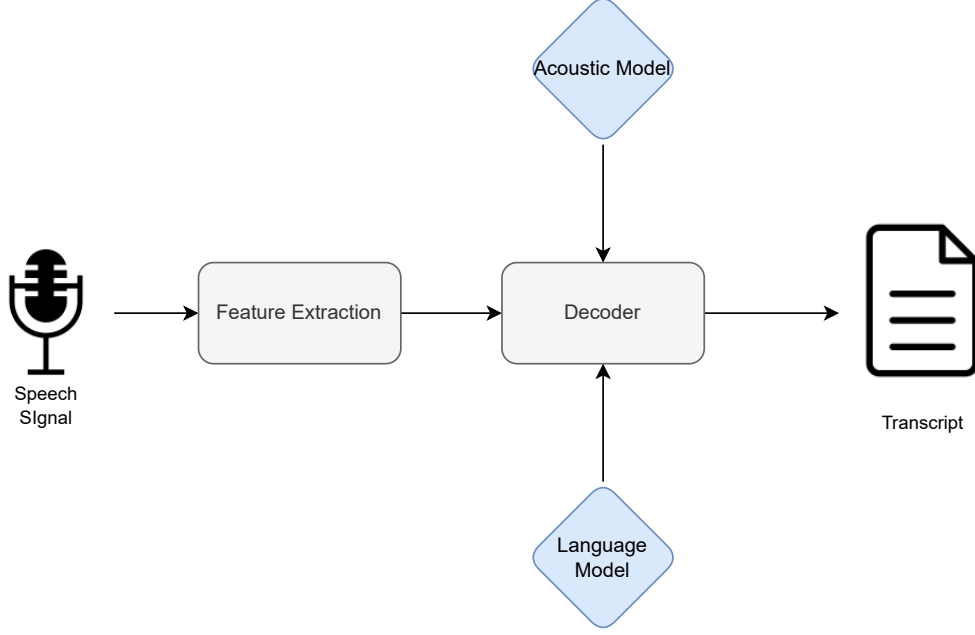


FIGURE 1.1: Block diagram of an ASR model

paradigm and the access to thousands of hours of speech data resulted in modern transformer models [14–16] that perform well on ASR tasks in many domains without using explicit language models. Nevertheless, the addition of an external language model to a neural ASR system remains beneficial, especially when the application domain lays outside the domains covered during pretraining. The integration of the external language model happens in two ways: either it reweighs the acoustic model’s predictions at each inference step – called first-pass decoding – or it reweighs alternative beams produced during first-pass decoding – called second-pass decoding.

Say we have a speech signal x , where the acoustic model makes a prediction for the time-frame corresponding to time t . This prediction is a causally conditional probability distribution across the vocabulary, which we can express as $p_{acoustic}(y_t|x, y_{t-1}, y_{t-2}, \dots, y_1)$. If we have an external language model p_{LM} that generates output probabilities conditioned on the string decoded so far, i.e., $p_{LM}(y_t|y_{t-1}, y_{t-2}, \dots, y_1)$ then at each step, we can combine these distributions as:

$$p_{ASR}(y_t|x) = \alpha p_{acoustic}(y_t|x, y_{t-1}, y_{t-2}, \dots, y_1) + \beta p_{LM}(y_t|y_{t-1}, y_{t-2}, \dots, y_1) \quad (1.1)$$

Where α and β are hyper-parameters that control the influence of the acoustic and linguistic modules. This is first-pass decoding. Second-pass decoding works a bit differently: during first-pass decoding, we maintain an N-best list of potential candidates by keeping track of N branches at each step. Then, we use a language model to score each of these branches and select the best one. This is often called lattice rescoring in the literature.

Note that the LM used for second-pass decoding scores only a very small fraction of the search space while the one used during first-pass decoding is applied at each inference step. This means that in practice, LMs used for second-pass decoding can be more computationally expensive than those used for first-pass decoding. Traditionally, this has resulted in the use of small n-gram models for first-pass decoding and larger neural models for second-pass decoding [17]. Although this setup is effective for general, everyday scenarios, things become more difficult when ASR systems have to be developed for domain-specific applications. The acoustic model does not warrant changes with changing domains: speech at a phoneme level is minimally influenced by topic or field. Language modeling on the other hand is much more sensitive: the LMs used for decoding need to be domain-specific to model linguistic components effectively. The simplest approach is to collect a large amount of text specific to the domain and create language models from it, but sometimes even the acquisition of such datasets becomes difficult. This is where pretrained language models (PLMs) become valuable: trained on massive and diverse text sources, PLMs inherently cover a much wider range of domains than pretrained speech models. As a result, they can help bridge the gap in linguistic coverage, compensating for the scarcity of high-quality in-domain text while still capturing domain-relevant patterns. The use of PLMs in the ASR setup described so far comes with two caveats:

1. Since PLMs are pretrained on thousands of domains, knowledge of the domain in question could be dormant or be catastrophically forgotten and might need revival. For example, given a random target domain, it is unsure if using PLMs out-of-the-box is optimal.
2. PLM use is restricted to second-pass decoding because of the computational bottleneck, which leaves first-pass decoding exposed to reduced domain knowledge.

We propose the use of continued pretraining and n-gram approximation (i.e., converting the PLM into an n-gram model) to tackle both issues: (1) Continued pretraining can resurface (and possibly enrich) any domain-specific knowledge in the PLM, which then makes it an ideal candidate for second-pass decoding (2) N-gram approximation strengthens the first-pass decoder through distillation and reduces the computational bottleneck at the same time.

1.1 Related Work

The potential of using PLMs in automatic speech recognition (ASR) is an area that has been insufficiently investigated so far. Within existing literature, several studies examine the use of PLMs in lattice rescoring [18–20], but their utilization in first-pass decoding remains under-explored. This is justified to some extent, since using PLMs directly in first-pass decoding would suffer from the same limitation as using RNNLMs, which is computational inefficiency [21, 22].

Despite the computational bottleneck, the use of RNNLMs in ASR has been studied extensively. In the context of first-pass decoding, two methods using RNNLMs have been explored, namely offline and online methods [23, 24]. The offline method involves approximating an RNNLM to generate an n-gram LM that is used for first-pass decoding and the online approach employs a cache to store the RNNLM states and prunes them during decoding [25]. Several offline approaches to approximate RNNLMs to n-grams have been explored previously [21, 23, 24, 26], but to the best of our knowledge, only the study by [27] looks at a similar approximation for PLMs. Their approach involves pre-training a GPT-like model on a very large general-domain corpus, fine-tuning it on a smaller domain-specific corpus, and then using it to generate a large corpus for approximation into an n-gram LM. They use pre-training corpora that have hundreds of thousands of sentences but approximation has not been explored in a low-resource scenario, where trends can vary significantly in comparison. Moreover, these approximation methods have not been studied for GPT-2, which is much larger and more powerful than GPT and has shown impressive few- and zero-shot capabilities [28] in comparison. Hence, this paper explores the use of GPT-2 in approximating and augmenting n-gram LMs under low-resource scenarios. Specifically, we compare text sampling-based conversion and probability-based conversion as a means of approximating GPT-2 in domain-specific settings. Our research contributions are:

1. Comparing the efficiency of text sampling and probability conversion in approximating GPT-2 into an n-gram model.
2. Introducing a vocabulary-restricted decoding method that improves text generation in domain-specific scenarios.
3. Evaluating the competitiveness of sampling-based approximation across domain difficulty and data scarcity.

The content presented in [chapter 2](#) and [chapter 3](#) is based on:

Aravind Krishnan, Jesujoba O. Alabi, and Dietrich Klakow (2023). *On the N-gram Approximation of Pre-trained Language Models*. In: *Interspeech 2023*, pp. 371–375. doi: [10.21437/Interspeech.2023-2182](https://doi.org/10.21437/Interspeech.2023-2182).

CHAPTER 2

APPROXIMATING PLMS INTO N-GRAMS

2.1 Data

We experiment with three datasets to span domain coverage, task difficulty, and resource availability. The sentence and word level distribution across all datasets is shown in [Table 2.1](#).

Taskmaster-2 is a spoken dialogue corpus [29] of 17,289 conversations between users and call-center operators. The data spans human conversations in six domains: restaurant-search, food-ordering, sports, music, movies, and flights. For our experiments, we combine all utterances in a call into a single sample. A train:dev:test split of 70:20:10 is used.

HUB4 1996 [30] is a broadcast news speech corpus that contains 104 hours of transcribed speech from television networks. The domains covered in broadcasts and interviews is broad, ranging from current affairs and politics to sports and international celebrations. To simulate a low-resource scenario, we use transcripts from the speech data (LDC97T22) for training and the standard development and test datasets (LDC97S66).

ATCO2 [31] is a corpus of air traffic communication between controllers and pilots. We use the 4-hour test set for training and the publicly released 1-hour test set for testing.

2.2 Approximating GPT-2

Existing literature supports the use of three offline methods for approximating RNNLMs into n-gram LMs: sampling-based approximation [21], probability-based approximation [26] and iterative approximation [32]. We concentrate on sampling and probability-based approximation methods for GPT-2 and briefly describe these methods in [subsection 2.2.1](#) and [subsection 2.2.2](#). We also present a vocabulary-restricted decoding scheme that controls the domain drift during text generation in [subsection 2.2.3](#).

Dataset	#Sentences / #Words		
	Train	Dev	Test
ATCO2	2337 / 27K	537 / 6K	826 / 10k
HUB4	46573 / 735K	4178 / 65K	346 / 19K
Movies	2139 / 344K	612 / 97K	305 / 49K
Restaurant-Search	2293 / 351k	656 / 98k	327 / 49k
Music	1122 / 142K	321 / 40K	160 / 19K
Flights	1736 / 332K	497 / 96K	248 / 47K
Sports	2436 / 279K	697 / 81K	348 / 39K
Food-Ordering	735 / 80K	210 / 22K	105 / 11K

TABLE 2.1: Dataset distribution and splits.

2.2.1 Sampling-Based Approximation

Given a background corpus B , sampling-based approximation (SBA) involves generating additional text samples from the same distribution as B using a language model such as an RNNLM or Transformer-based LM. The generated text is used to train a n -gram LM, which approximates the probability distribution of the neural model itself [21]. The performance of this approach is often improved by interpolating the approximated n -gram LM with a baseline n -gram LM [23, 26] trained on B . This approach has been studied with RNNLMs [21, 23, 26] and Transformer-based LMs [27] but has not been explored with massively pre-trained PLMs. In this work, we study how SBA performs with GPT-2 across datasets, data volume and domains.

2.2.2 Probability-Based Approximation

We also explore the use of probability-based approximation (PBA) for GPT-2, which was initially proposed by [26] as a way to approximate RNNLMs into n -gram LMs. Instead of using count-based probabilities, this method involves extracting and assigning n -gram probabilities directly from trained RNNLMs. Since GPT-2 uses subword (BPE) tokens, we obtain word-level probabilities by multiplying the conditional probabilities of each BPE token in the target word. For a word W composed of BPEs $\{w_1, w_2 \dots w_n\}$, the word level probability is computed as:

$$p(W|H) = \prod_{i=1}^n p_{gpt2}(w_i|w_{i-1} \dots w_1 H) \quad (2.1)$$

Where H is the sentence history and p_{gpt2} is the conditional probability assigned to a BPE token by the fine-tuned GPT-2. The conditional word probabilities are reassigned to n -gram probabilities and averaged according to the original setup. Backing off is done from history sums taken from a trigram LM developed from the training corpus. The authors in [26] observe

that the resulting model produces the best perplexities when unigram probabilities are borrowed from the baseline trigram LM.

2.2.3 Vocabulary-Restricted Decoding

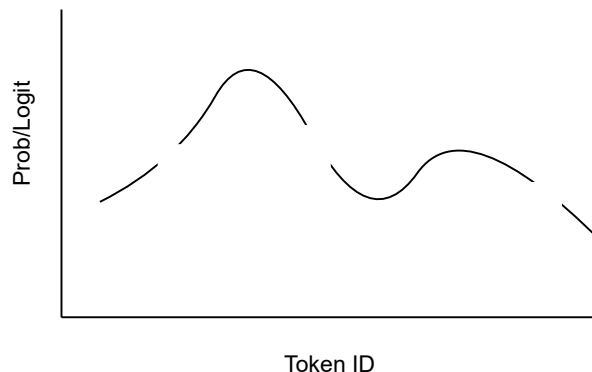


FIGURE 2.1: Illustrating the manipulation of the output logits for Vocabulary restriction. Probabilities for unwanted BPE tokens are made to vanish manually and the distribution is re-normalized to predict from the desired subset.

We also propose a “vocabulary-constrained” extension to SBA: We restrict the BPE tokens that can be generated to the BPE vocabulary of the training set. This restriction is motivated by the fact that the BPE vocabulary of any PLM spans thousands of characters irrelevant to any specific domain (and can be omitted). During random sampling, a considerable amount of probability mass is lost across this long tail, which could be otherwise preserved and redistributed among relevant tokens. We start by creating a BPE vocabulary over all words in the training data. Each word is tokenized ¹ and all composite BPEs are added to the BPE vocabulary. During random sampling, softmax operations are performed only over the BPEs in our vocabulary. The sampling algorithm is thereby restricted to pick from the BPEs present in this vocabulary. We call this Vocabulary-Restricted Decoding. Samples of text constructed with this decoding setup will be restricted to words from our vocabulary. We distinguish our decoding scheme from lexically constrained decoding [33, 34], where restrictions are placed on *including* specific tokens during generation without restrictions on other tokens that can co-appear (ie., word w has to appear in the decoded sentence). Our approach restricts the vocabulary space in its entirety during decoding, *excluding* non-relevant tokens entirely. Figure 2.2 illustrates this difference. We expect that in domain-specific scenarios, constraining the generation vocabulary will suppress unwanted tokens and force GPT-2 to model the words in the training set better.

¹GPT-2 tokenizes words with and without preceding blanks (“apple” and “_apple”) differently. We add both versions into the vocabulary

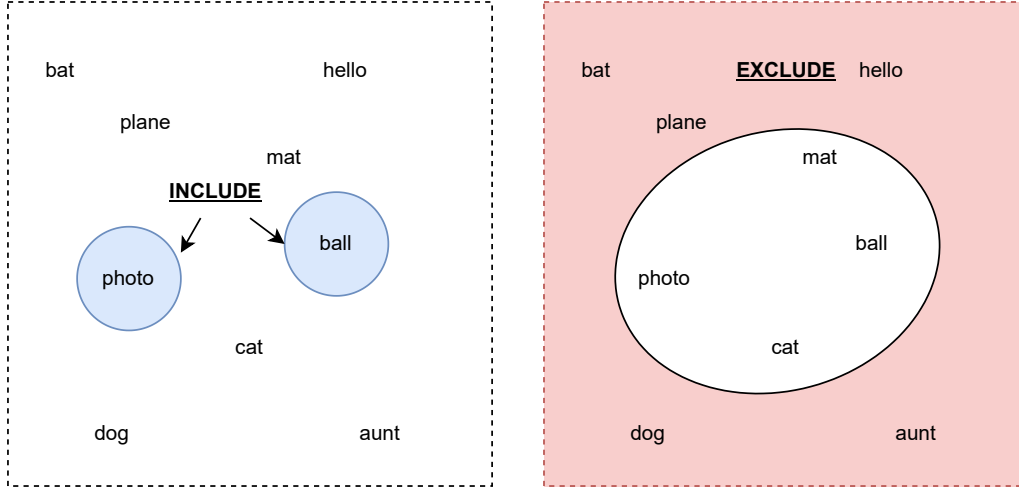


FIGURE 2.2: Illustrating the differences between Vocabulary restriction (right) and Lexical restriction (left) on a sample vocabulary space

2.3 Experimental Setup

We use the 124M parameter version of GPT-2 from *huggingface* [35] for all experiments. The model is fine-tuned with a learning rate of $5e-5$ and a weight decay of 0.01. The learning rate is linearly warmed up using $\min(\#train\ samples, 100)$ steps. Training is terminated using early stopping on the development set with patience of 5. Text generation uses top- p [36] sampling with $p=0.95$ and a temperature of 1.0. Unless specified otherwise, we generate 100 times the training data for all SBA experiments. The n-gram models are trigram models with Kneser-Ney smoothing [5], and are built and evaluated with the SRILM [37] Toolkit.

In the following sections, we use acronyms to denote trained n-gram models for the sake of brevity: **KN3** denotes the baseline model trained on just the training corpora; **RS-KN3** is trained on the random-sampled corpora and **VR-KN3** on corpora sampled using vocabulary-restricted sampling. Note that RS-KN3 and VR-KN3 are both SBA approaches with different decoding schema. For ease of comparison, a common vocabulary is constructed for all experiments by combining the vocabularies of the training corpus and the 100x corpora generated for RS-KN3 and VR-KN3². This results in an OOV rate of less than 2% for each dataset.

2.4 Comparing Probability and Sampling Approximation

In this section, we compare the two approaches used for n-gram approximation: PBA and SBA. More specifically, we ask the following question:

²Except in section 2.4 where the test vocabulary is additionally introduced

Dataset	GPT-2	RS-KN3	PBA
ATCO2	25.04	47.04	80.04
HUB4	189.91	369.90	632.92
Movies	13.53	27.92	49.89
Restaurant-Search	13.26	28.39	46.60
Music	11.55	35.23	57.83
Flights	7.23	17.83	18.10
Sports	12.77	13.46	30.74
Food-Ordering	9.96	10.37	23.31

TABLE 2.2: Sampling-based approximation of GPT-2 into an n-gram model consistently outperforms probability-based approximation

What is the degradation in perplexity incurred when approximating a PLM into an n-gram using PBA or SBA?

To measure degradation, we compare the test set perplexities of PBA and SBA models with the “true” perplexity of a fine-tuned GPT-2. For computing the “true” perplexity, word level perplexities are obtained from GPT-2 using the computation described in Mielke [38] and elaborated in Cotterell et al. [39]. To simplify perplexity comparisons, we add the test vocabulary to the n-gram models, thus ensuring zero OOV test rate. Results are presented in Table 2.2.

SBA outperforms PBA in perplexity degradation Except for *sports* and *food-ordering*, both approximation methods incur at-least 100% degradation when compared to the “true” perplexity. In comparison, previous literature that looked at perplexity differences associated with PBA approximation in RNNLMs report Singh et al. [23], Adel et al. [26] less drastic degradations. The worsening degradation could be explained by PLMs’ tendency to overestimate n-gram probabilities in comparison to RNNLMs. A closer look confirms that GPT-2 overestimates lower-order n-gram probabilities during PBA because of probability sharing between higher and lower-order n-grams. In all cases however, we see that the degradation associated with SBA is much lesser than that seen for PBA. It is to be noted that the vocabulary set for GPT-2 is BPE based and is more fine-grained than the word-level model. We can therefore only assume that our computation for the non-approximated GPT-2 perplexity underestimates the true metric. Nevertheless, as the underestimated metric outperforms the word-based model, the granularity of the vocabulary does not interfere with our finding.

We conclude that PBA is a lossy conversion scheme compared to SBA and focus on SBA for the remaining part.

Model	Movies	Restaurant	Music	Flights
KN3	22.31 / 1210	23.01 / 1404	34.41 / 1269	19.50 / 556
RS-KN3	25.67 / 1004	25.09 / 1241	31.11 / 639	17.59 / 397
VR-KN3	23.54 / 980	23.34 / 1066	28.41 / 969	16.47 / 404
KN3 + RS-KN3	19.58	20.26	26.03	15.96
KN3 + VR-KN3	19.12	19.71	25.47	15.70
RS-KN3 + VR-KN3	25.92	22.96	26.64	16.34
Total Interpolation	18.91 / 795	19.54 / 955	24.38 / 440	15.53 / 249

TABLE 2.3: Perplexities and OOV rates for the n-gram models (Movies, Restaurant, Music, Flights). Metrics are *Perplexity*/*#OOVs*.

Model	Sports	Food	ATCO2	HUB4
KN3	13.24 / 639	12.62 / 296	31.95 / 428	275.19 / 517
RS-KN3	12.77 / 605	9.96 / 119	44.00 / 446	310.19 / 469
VR-KN3	12.35 / 593	9.91 / 260	35.53 / 445	342.72 / 452
KN3 + RS-KN3	11.45	9.33	29.27	207.16
KN3 + VR-KN3	11.38	9.53	28.60	238.21
RS-KN3 + VR-KN3	12.19	9.49	35.21	241.74
Total Interpolation	11.29 / 583	9.10 / 98	28.54 / 415	207.32 / 337

TABLE 2.4: Perplexities and OOV rates for the n-gram models (Sports, Food, ATCO2, HUB4). Metrics are *Perplexity*/*#OOVs*.

2.5 Vocabulary Restriction and Interpolation

In this section, we compare the flavors of SBA that we propose by comparing the baseline n-gram models with the VS-KN3 and RS-KN3 models. Results are tabulated in Table 2.3 and Table 2.4. We summarize the main findings below

Sampling reduces OOV rates Except for ATCO2, both sampling methods produce an OOV reduction on the test set. Combining vocabularies from the original corpus and the sampled corpora reduces the OOV count by more than 30% for six datasets. A closer look at the OOVs contributed shows that vocabulary-restricted decoding caters to affixing and derivational inflections while random sampling has additional coverage over novel nouns and verbs.

Sampled LMs outperform baseline In relative terms, we see that RS-KN3 under-performs the baseline considerably (+40% ppl) with ATCO2, where we observe that the unconventional grammatical structure adversely affects sampling. On the other hand, VR-KN3 performs strongly here with 20% improvement over KN3. This suggests that controlling the drift of random sampling is beneficial in tightly constrained domains. In general, we see that vocabulary restriction improves over random sampling when the domain (and consequently the vocabulary) is restricted. However, performance suffers with HUB4, where the domain is wide-ranging and

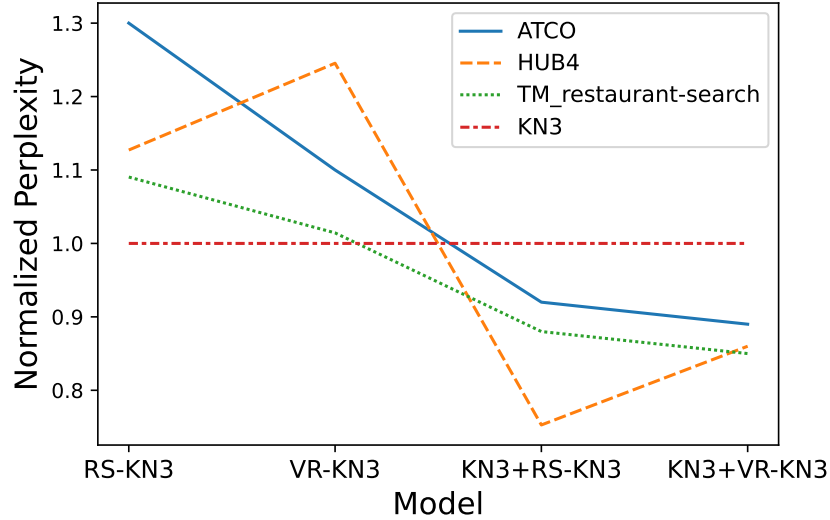


FIGURE 2.3: Interpolation with sampled n-gram models reduce baseline perplexity across all datasets. All perplexities normalized against the respective KN3

the vocabulary cannot be restricted as dramatically. Vocabulary restriction also outperforms random sampling for all datasets in Taskmaster, where a satisfactory equilibrium exists between grammatical complexity and domain restriction.

Interpolation yields strong models In all cases, an improvement over the baseline is observed as soon as either of the sampling methods is interpolated with the baseline KN3³. Here again, domain specificity plays its part, and interpolation with VR-KN3 outperforms RS-KN3 in the case of ATCO2 while the latter performs strongly for HUB4. Interpolation with either sampled model shows promising improvement with Taskmaster. We observe the best performance across all datasets when all three models are interpolated together. On average, interpolating the baseline with RS-KN3 gives a 15% reduction in test perplexity and additional interpolation with VR-KN3 decreases perplexity further by 5%.

We conclude that interpolation with a large-scale sampled corpus is almost always beneficial for a trigram LM and is a simple and effective method to exploit PLMs for first-pass decoding.

³interpolation weights are tuned with the respective development sets

CHAPTER 3

FACTORS AFFECTING SAMPLING BASED APPROXIMATION

We devote this section to answering the following question -

Can SBA LMs outperform the baseline KN3 without interpolation?

We isolate and study two variables that influence augmentation: the domain of the dataset and the number of training samples available.

3.0.1 Domain difficulty

Comparing language modeling difficulty across two domains is challenging in a statistical setup: the differences in vocabulary and the lack of standardized background corpora means that any change in perplexity cannot be easily traced back to the domain itself. Sub-word modeling and pre-training mitigate these concerns and PLMs prove to be ideal sandboxes to compare difficulties in modeling different domains. Assuming that pre-training acquaints PLMs with everyday words, the adaptability of a domain over fine-tuning can then approximate the difficulty in modeling it. We quantify it by computing the dev set perplexities of fine-tuned¹ GPT-2 models and compare them across domains to rank their difficulty. Note that this comparison bypasses the machinery we used in Section [section 2.4](#) since comparisons are made within the PLM space.

[Figure 3.1](#) shows the trend in test perplexity with an increase in the number of sampled sentences for RS-KN3. The figure shows that HUB4 and ATCO2 are both difficult for GPT-2 to model, but HUB4’s steeper slope suggests a preference for additional sampling when domains are spread thin. In most cases, we see that an increase in the number of samples generated correlates with a decrease in test perplexity. Larger sampling has a slower effect on perplexity after a certain

¹Perplexity from a pre-trained model could be misleading since the domain might look hard initially but be easily learnable.

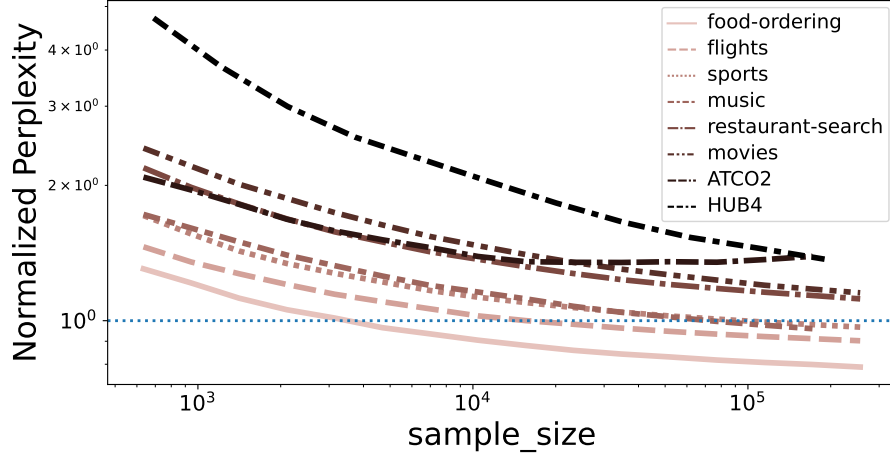


FIGURE 3.1: Benefits of sampling depends on domain difficulty. Perplexities are normalized across the respective KN3, which is shown as the horizontal line at 1.0. Darker colour denotes higher perplexity on the dev-set

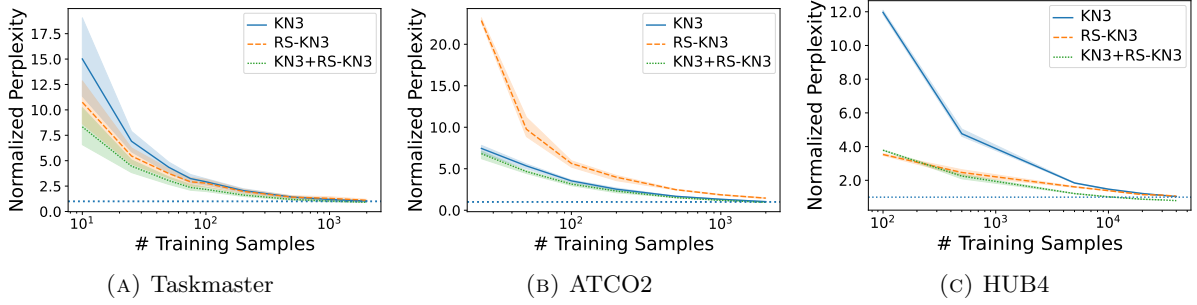


FIGURE 3.2: Performance of language models across sub-sampled training corpora. Perplexities are normalized with the full-scale KN3 perplexity (which is shown as the blue dotted line at 1.0). The Taskmaster datasets are averaged to generate one plot.

point and produces long tails afterward. For datasets that have a lower dev set perplexity, SBA is seen to beat the baseline KN3 early on. An increase in domain difficulty stretches the cross-over point and we see that the number of sentences required to outmatch the baseline is proportional to the difficulty of the domain. We observe similar trends for VR-KN3 models as well, with a slight downward shift along the y-axis for all curves (except for HUB4, where the shift is slightly upwards). This analysis provides us with two conclusions:

1. *Generating additional data correlates with reductions in test perplexity, but*
2. *The amount of generated data required to match the baseline trigram model has an inverse relation with domain difficulty.*

3.0.2 Few-Shot Approximation

This section compares the responses of SBA and the baseline KN3 under low-resource settings. We simulate few-shot scenarios by sub-sampling the train and development corpora while keeping the test corpus unchanged. For each setting, GPT-2 is fine-tuned on the sub-sampled training set, using the sub-sampled development set for early stopping. The fine-tuned model then generates 100x the size of the sub-sampled train set. The resulting perplexities are normalized with the full-scale KN3 value for each dataset. Each experiment is run across three random seeds. The results are plotted in [Figure 3.2](#). The trends for ATCO2 reflect the domain difficulty yet again, with GPT-2 struggling to understand the domain in few-shot settings and the trigram model performing significantly better. GPT-2 does not outperform the trigram corpus in any data scenario, and the interpolated model hugs the KN3 curve. For Taskmaster and HUB4 on the other hand, the generated corpus fares significantly better in few-shot scenarios. The perplexity of the generated corpus is several times lower than the sub-sampled KN3 model when the number of samples available is less than 100. This can be attributed to large-scale pre-training, which helps the PLM generalize text quicker than a KN3 under low resource settings. However, the KN3 curve has a steeper decline, which suggests that it benefits more from the availability of additional curated corpora than a PLM in few-shot scenarios.

We also see that the improvements produced with the additional 100x data become decreasingly pronounced with increasing train data size. i.e., once the training data is large enough to generalize the domain well, the baseline KN3 becomes a tough competitor. For HUB4, the sub-sampled KN3 even outperforms the generated corpus for train sets larger than 20K samples. Improvements can be made by generating exponentially many samples even at “high” resource settings, but the cost-benefit ratio is significantly more encouraging in few-shot settings. This is in contrast to [24], where larger training data resulted in larger gains for word-based RNNLM models. Our conclusions are as follows:

1. *In domain-specific settings, PLMs show strong generative abilities even under few-shot conditions.*
2. *However, in higher-resourced scenarios, n-gram models remain strong baselines.*

CHAPTER 4

ACOUSTIC TRANSFORMERS IN SPEECH

In [chapter 1](#), we saw the statistical ASR setup and its sub-modules. We saw how pretrained, transformer based **language** models are strong linguistic components for ASR and saw how they can be integrated into first-pass decoding. In this section, we are going to look at pretrained transformer models again, except now at pretrained **acoustic** transformer models.

We mentioned in [chapter 1](#) that earlier attempts at acoustic modeling using neural networks focused on predicting HMM states [7] or CTC based decoding using end to end training. This approach changed with the advent of the pretraining-finetuning paradigm [13], which introduced the development of “general-purpose” encoders for natural language modeling. The central idea in this paradigm is to use large volumes of unlabeled data and self supervised-losses (along with some architectural innovations) to arrive at strong representations of written text. Naturally, an equivalent problem arose in speech – ‘Can we pretrain large acoustic models on vast amounts of unlabeled speech data and then fine-tune them for specific downstream tasks like ASR, speaker recognition, or emotion classification?’. This question sparked a wave of research into self-supervised acoustic pretraining approaches in the early 2020s. It resulted in the development of speech-SSL models, which learn rich acoustic representations without requiring frame-level supervision. These models improved both the robustness and the sample efficiency of downstream systems, much like pretrained language models revolutionized NLP. Over a short period of time, self supervised (SSL) models for speech achieved state of the art performances for many downstream speech tasks like ASR, SID, etc [40].

SSL Training Different SSL models vary in their strategies for speech processing and loss computation but an underlying training pipeline is as follows: Given an Acoustic input $X = \{x_1, x_2 \dots x_T\}$, a feature extractor $f(\cdot)$ extracts features from the raw signal, generating $F = \{F_1, F_2 \dots F_{T^*}\}$ where $T^* < T$. A masking operation $m(\cdot) \in [0, 1]$ is applied on top of the feature embeddings, where some embeddings are masked from the subsequent models. A transformer block $\theta(\cdot)$ then processes these masked features $m(F)$, producing a set of *contextualized* (via the attention mechanism) embeddings $H = \{H_1, H_2 \dots H_{T^*}\}$. To compute labels for supervision,

a quantization module $q(\cdot)$ is used, which converts X into quantized targets, i.e., $q(X) = Q$, where $Q = \{Q_1, Q_2 \dots Q_{T^*}\}$. Losses are computed between the masked positions of H and Q , and are back-propagated through the transformer (and optionally the quantization module). We illustrate this pipeline in [Figure 4.1](#) and discuss some popular SSL models below.

Wav2Vec2.0 [41] employs a contrastive objective, distinguishing the true masked representation from a set of distractors. Let’s assume that position m is masked by the masking operation. The transformer generates hidden representations $H = \{H_1, H_2 \dots H_m \dots H_{T^*}\}$. Quantization is done using code-book vectors and Gumbel-Softmax [42], yielding label representations $Q = \{Q_1, Q_2 \dots Q_m \dots Q_{T^*}\}$. Wav2vec2 training involves bringing closer representations (H_m, Q_m) while bringing apart all other $(H_i, Q_i), i \neq m$. i.e., it implements a contrastive loss where the distractors are sourced from other positions in the signal.

$$\mathcal{L}_{\text{con}} = - \sum_{m \in \mathcal{M}} \log \frac{\exp(\text{sim}(H_m, Q_m)/\kappa)}{\sum_{q \in \{Q_m\} \cup \mathcal{N}_m} \exp(\text{sim}(H_m, q)/\kappa)}, \quad (4.1)$$

where $\text{sim}(\cdot, \cdot)$ denotes cosine similarity and κ is a temperature parameter.

To avoid *codebook collapse*, i.e., only a few codewords being used, Wav2Vec2.0 further employs a *diversity loss* that encourages uniform codeword usage. Let G denote the number of codebook groups and V the number of entries per group. If $p_{g,v}$ is the empirical probability of selecting entry v in group g , the diversity loss is defined as:

$$\mathcal{L}_{\text{div}} = \frac{1}{GV} \sum_{g=1}^G \sum_{v=1}^V p_{g,v} \log p_{g,v}. \quad (4.2)$$

The overall pretraining objective is then a weighted sum of the two terms: $\mathcal{L} = \mathcal{L}_{\text{con}} + \alpha \mathcal{L}_{\text{div}}$ where α controls the strength of the diversity regularization. **FastRQ** [43] improves upon Wav2Vec2 by using mel-spectrograms as input and a randomly initialized vector quantizer as the quantization module. This is seen to improve training speed by 50% [44] while maintaining the original performance of Wav2vec2.

HuBERT [15] implements a predictive loss at masked positions, using clustered labels from quantized representations. As before, masked representations $H = \{H_1, H_2 \dots H_m \dots H_{T^*}\}$ are generated from the transformer model with randomly masked positions. Quantization is done by clustering MFCC representations of the speech signal (on the intermediate representations in later epochs), by running K-means clustering on $MFCC(X) = \{M_1, M_2 \dots M_m \dots M_{T^*}\}$ and obtaining $C = c_1, c_2 \dots c_m \dots c_{T^*}$ where c_i is the K-means label for M_i . For training, we maximize the probability that the hidden representation at the masked position H_m gets the

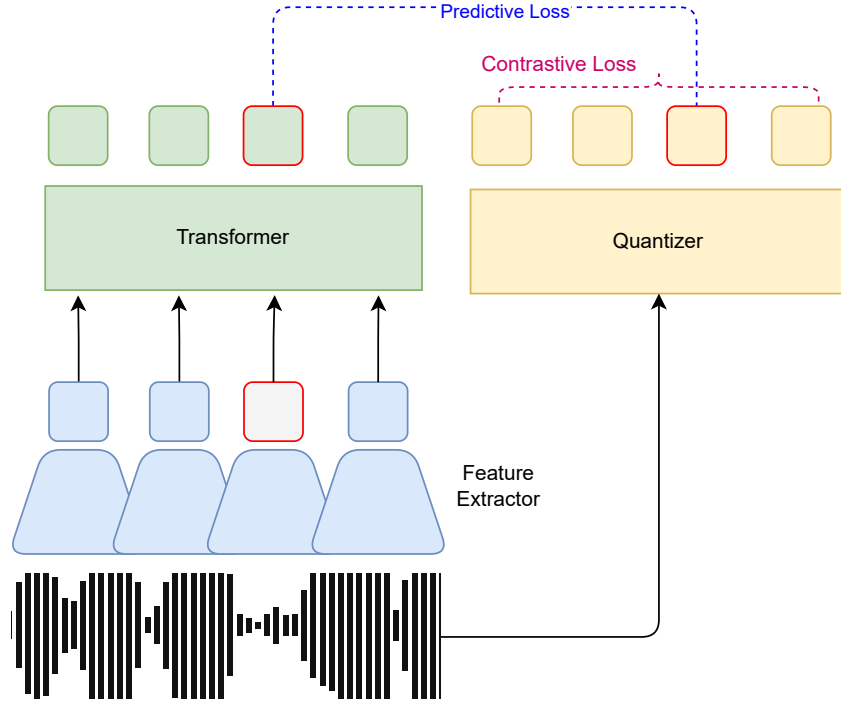


FIGURE 4.1: SSL pretraining architecture

same cluster number as that of c_m . i.e.,

$$L_m = -\log p(c_m | H_m) \quad (4.3)$$

$$p(c_m | H_m) = \frac{\exp(\text{sim}(H_m W_P, e_{c_m})/\tau)}{\sum_{c'=1}^C \exp(\text{sim}(H_m W_P, e_{c'})/\tau)} \quad (4.4)$$

where W_P is a projection matrix, e_{c_m} is the embedding for codeword c_m , $\text{sim}(a, b)$ computes the cosine similarity and τ scales the logit. **WavLM** [45] builds on HuBERT but introduces denoising pretraining, in which part of the input signal is replaced with noise or speech from other speakers. The target is to then predict the pseudo-labels of the original speech on the masked region, which teaches the transformer to become robust to real-world acoustic conditions. Despite these differences, the common thread is that the pretraining stage yields contextualized speech embeddings that can later be fine-tuned with modest amounts of labeled data, resulting in state-of-the-art performance across ASR and a broad range of paralinguistic tasks. The reader is directed to Mohamed et al. [46] for a broad survey on SSL architectures.

SSL Interpretability Despite their success, SSL speech models are often treated as black boxes, prompting efforts to analyze the internal representations they learn and the extent to which these align with linguistic or speaker-related information. Early work in this direction [47] showed that the convolutional layers that extracted features were extracting spectrograms

from the signal, and that the transformer block was encoding -phonetic [48] and word-level [49] information in progressive layers. Subsequent work showed that SSL representations do not strongly encode higher-order linguistic phenomena well [50]. It has also been shown that these representations strongly encoded speaker characteristics [51, 52], making them useful for tasks like speaker verification and identification [40].

4.1 Gender and SSL models

In the previous section, we discussed that the advent of transformer-backed speech models has had a dramatic effect on speech processing, with impressive results on several downstream benchmarks. Although we marvel at their "performance", recent research [53] has questioned the inclusivity of ASR models, holding them accountable for propagating (mostly) human biases. One direction in uncovering model bias is on the basis of socio-linguistic attributes [54]. Gender has been a very popular attribute among them, with several works suggesting a bias towards male [55] or female [53, 56–59] speakers. Apart from ASR models, these trends have also been reported in speech translators and speech classifiers [60–62].

Most works that study how gender influences a speech model’s prediction quantify it using performance differences between speaker genders. Although such performance disparities act as a litmus test for detecting bias, more analysis is needed to understand *how* these models encode and utilize gender in the first place. The closest work that explores this direction is [53], where the authors uncover gender embedded in the final layer of Whisper’s encoder. In this work, we study the encoding and the use of gender across the layers of two transformer-based ASR models. We expand our investigation by exploring if the detection of an attribute in the hidden layers of a model ensures its utilization by the model. In amnesic probing [63], the authors assert that utilization can only be guaranteed by elimination, i.e., Removing a property the model uses will disrupt it more than removing one it does not use. The authors show that the removal of semantic and syntactic properties affect BERT [64] differently. For removing an attribute from an embedding space, they employ Iterative Null-space Projection (INLP) [65], where data points are projected onto the null space of the linear boundary that separates them into the classes of the target property. Note that both these methods work in the linear domain, hiding the target attribute from any *linear classifier* that tries to uncover it from the embeddings.

This work proposes an experimental approach based on amnesic probing to study the encoding of gender in ASR models. Formally, we ask the following question: “***How much gender-related information can we linearly erase from an SSL model and what are the implications of linear erasure?***”. Following [63, 65, 66], we confine ourselves to the linear domain, probing and removing linearly encoded gender from the model’s hidden layer representations. To the

best of our knowledge, this is the first work that employs linear erasure to analyze and anonymize ASR embeddings. Our contributions are as follows:

1. We effectively remove linearly encoded gender information across all layers of the Wav2Vec2 and HuBERT models, without necessitating model training (§5.3.1).
2. We analyze the effects of linear erasure on downstream ASR performance (§5.3.2).
3. We probe for the frame-level organization of gender across model layers (§6.0.2).

The content presented in chapter 5 and chapter 6 is based on:

Aravind Krishnan, Badr M. Abdullah, and Dietrich Klakow (2024). *On the Encoding of Gender in Transformer-based ASR Representations*. In: *Interspeech 2024*, pp. 3090–3094. doi: [10.21437/Interspeech.2024-2209](https://doi.org/10.21437/Interspeech.2024-2209).

CHAPTER 5

GENDER ERASURE

In this section, we progressively remove gender from SSL representations using linear erasure. Linear erasure is a method used to linearly remove a target attribute encoded in a set of vectors. Given a set of vectors $X = [x_1, \dots, x_n]$, $x_i \in \mathbb{R}^d$ that associate to a set of target attributes $Z = [z_1, \dots, z_n]$, $z_i \in \{1, \dots, k\}$, linear erasure entails learning a transformation $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ such that we can no longer train **linear classifiers** to predict z_i from $f(x_i)$. When this happens, we say that Z has been linearly erased from X . A vital objective in choosing f is to make sure that the transformed embeddings retain all information except the target attribute.

While there has been several attempts at finding an efficient erasure function [65, 67], they do not guarantee erasure or hurt the original embedding space non-trivially. The authors of [66] propose Least Squares Concept Erasure (LEACE), a closed form solution to linear erasure. They begin by proving that if the class centroids of every class in Z coalesce, it implies linear erasure of Z from X . They then derive the transformation that ensures class centroid equality with minimal impact to the original embeddings (impact is measured in terms of mean squared norm). They arrive at the LEACE transformation for an embedding $\mathbf{x}$ as:

$$f_{\text{LEACE}}(\mathbf{x}) = \mathbf{x} - \mathbf{W}^+ \mathbf{P}_{\mathbf{W}\Sigma_{XZ}} \mathbf{W}(\mathbf{x} - \mathbb{E}[X]) \quad (5.1)$$

where Σ_{XZ} is the covariance between matrices X and Z , $\mathbf{W}$ is the whitening transformation $(\Sigma_{XX}^{1/2})^+$ and W^+ denotes Moore-Penrose pseudoinverse of W . $\mathbf{P}_{\mathbf{W}\Sigma_{XZ}} = (\mathbf{W}\Sigma_{XZ})(\mathbf{W}\Sigma_{XZ})^+$ is the orthogonal projection matrix onto $\text{colsp}(\mathbf{W}\Sigma_{XZ})$. In practice, LEACE whitens $\mathbf{x}$, projects it into the correlation subspace between X and Z and then un-whitens the result. This is then subtracted from $\mathbf{x}$, which guarantees linear erasure of Z . This process is shown in [Figure 5.1](#).

5.0.1 Concept Scrubbing

In [66], the authors introduce concept scrubbing, a way to (linearly) remove a selected attribute from *all* layers of a transformer. Concept scrubbing is done by linearly erasing the attribute

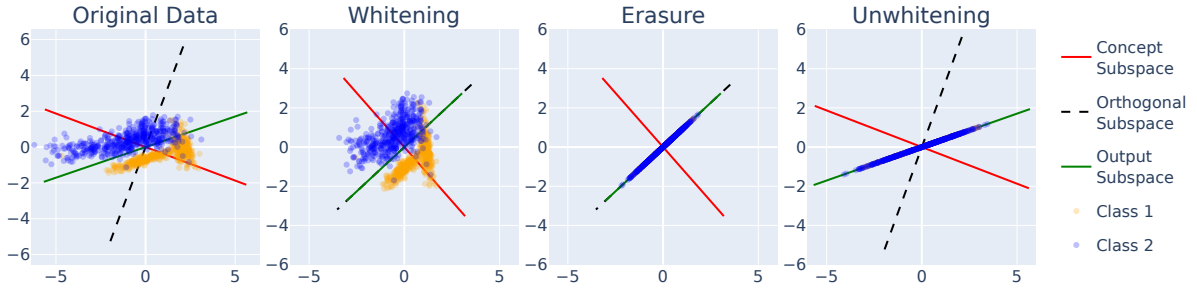


FIGURE 5.1: Steps used for LEACE. Figure sourced from Belrose et al. [66]

Dataset	# Utterances		#Male/#Female Speakers	
	Train	Test	Train	Test
TIMIT	4620	1680	326/136	112/56
Librispeech	28539	2620	126/125	20/20

TABLE 5.1: Dataset statistics for TIMIT and Librispeech. Note that there is no speaker overlap between the train and test sets

from each output embedding in the transformer, before it is fed into the next layer. Scrubbing cannot be done in parallel, since any intervention at layer L of the transformer affects the hidden states at every subsequent layer $L' > L$. Therefore, linear erasure is done in cascade, beginning at the first layer and sequentially proceeding to the final layer of the model. LEACE makes it easy to implement concept scrubbing since the parameters can be tuned in a streaming fashion, which eliminates the need of storing hidden states in memory or on disk.

5.1 Data

TIMIT [68] is a read speech corpus of 630 speakers of various American dialects. Each speaker in the corpus reads 10 phonetically rich sentences in English, intended to capture a wide range of phonetic phenomena. TIMIT provides the reported gender of each speaker in the corpus, which is one of Female or Male. We use the default train/test splits for all experiments.

Librispeech [69] is a speech corpus of about 1000 hours of read audio-books. Gender is reported for every utterance and is again binary. We use the clean 100 hour training set and the default test set for all probing experiments. Note that the ASR variants of Wav2Vec2 and HuBERT are both fine-tuned using a 960 hour super-set of Librispeech (excluding the test set).

The combination of Librispeech and TIMIT offers an in-domain and out-of-domain scenario for experiments. The speaker and gender distribution for the datasets is shown in Table 5.1.

5.2 Experimental Setup

All experiments are run on wav2vec2-large-960h¹ and hubert-large-ls960-ft², which are ASR fine-tuned versions of Wav2Vec2 and HuBERT respectively. Both models have been fine-tuned using the 960 hour Librispeech dataset with a CTC objective [6]. **Throughout this work, the models are kept frozen and set to evaluation mode.** The linear probes trained are SGDClassifier models from the *sklearn* [70] library. The non-linear probe trained in section 5.3 is an multi-layer perceptron classifier from *sklearn* with default parameters. All probes are trained on three random seeds until the training loss converges. Concept scrubbing is deterministic since the model is frozen, the train set is fixed and the solution is closed-form.

5.3 Gender Scrubbing

In this section, we employ the concept scrubbing method described in subsection 5.0.1 across the layers of two ASR models. We ask the following question:

Is gender scrubbing from ASR models possible? If so, what are its effects on downstream WER?

Let us start by assuming an audio corpus $\mathbf{X}$. Each audio sample $x_i \in X$ is associated with a gender label $z_i \in \{Female, Male\}$. When x_i is fed into an ASR transformer, each encoder layer L_j produces H dimensional latent representations across T_i time steps of x_i , generating an embedding of size (T_i, H) . We take the mean across the temporal dimension T to obtain a single representation for each utterance. Then, we train linear classifier to predict the target attribute from the mean-pooled utterance representations. We call this **Mean Probing**. The performance of the probe is used as a measure of the gender information available at this layer. To erase gender information at layer L_j , we train a LEACE eraser E_j over T_i embeddings using z_i as target label. The eraser is trained over every data point in X in a similar fashion. Once training finishes, we loop through the entire dataset again, erasing the gender information using the trained E_j . These *gender-erased* embeddings are now fed into the next layer L_{j+1} , and the process is repeated until the last transformer layer. Sequential erasure ensures that no layer of the transformer network has access to linearly encoded gender.

¹<https://huggingface.co/facebook/wav2vec2-large-960h>

²<https://huggingface.co/facebook/hubert-large-ls960-ft>

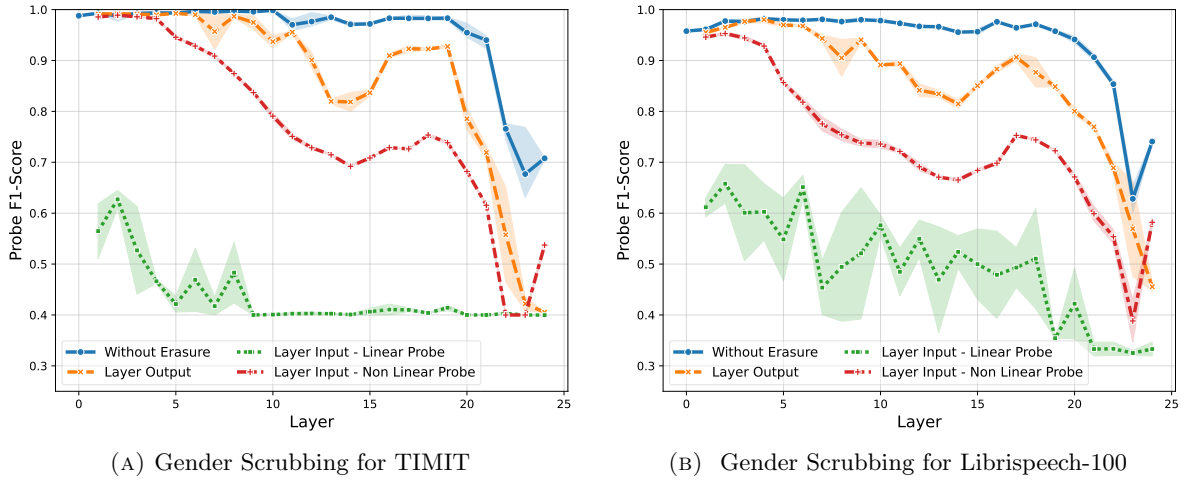


FIGURE 5.2: Gender scrubbing for the Wav2Vec2 ASR model. Plots depict linear probe performances at the input and output when linearly erasing gender from the input at each layer. Mean probe performance on the original model is also shown.

5.3.1 Tracking Erasure

To monitor the traversal of encoded gender along the model, linear probes are trained at the input and the output of each layer. A performance difference between the input/output probes at any layer can then be attributed to the operations of the layer itself. Note that the input at each layer is the gender-erased output from the previous layer. The input/output probe performances for TIMIT and Librispeech-100h (using Wav2Vec2) is visualized ³ in Figure 5.2. For both datasets, we see that the linear probes trained at the input have a near-random performance, which suggests that the erasure operation is successful at each layer. However, the results for the output probe show that the model recovers gender almost entirely in the early layers. This recovery can be attributed to two factors:

1. The non-linearity available to the transformer layer
2. The parametric capacity of the layer itself

To isolate the contribution of each of these factors to the probing trends, we train a simple non-linear probe on the linearly-erased input embeddings. The non-linear probe is a multi layer perceptron trained with softmax operation. The absolute performance of the non-linear classifier will then reveal any non-linearly encoded gender that lingers after linear erasure. In Figure 5.2 and Figure 5.3, we see that for the linearly scrubbed embeddings at the input, the non-linear probe shows high performance. This shows that the erased representations still contain non-linearly encoded gender, especially in the early layers of the model. The performance gap

³The reader is encouraged to view complete set of plots in this link: https://github.com/Krishnan-Aravind/Interspeech_2024_Gender

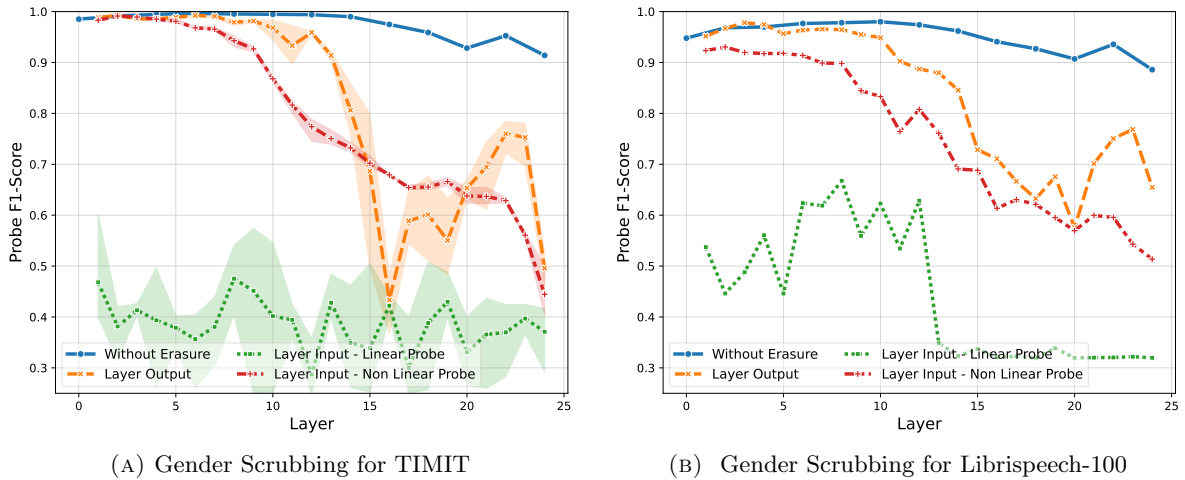


FIGURE 5.3: Gender scrubbing for the HuBERT ASR model. Plots depict linear probe performances at the input and output when linearly erasing gender from the input at each layer. Mean probe performance on the original model is also shown.

between the non-linear input probe and the output probe from the model is attributed to the parametric capacity of the layer, i.e., the model has more non-linearity than our probes. This means that the model recovers gender information even after scrubbing because it is still encoded non-linearly. However, non-linearly encoded gender in the scrubbed inputs and the consequent recovery of gender information is seen to weaken in the latter layers. The difference between the non linear probe and the output probe declines sharply after layer 16, where one also observes a prominently decreased recovery capacity. This suggests that in the latter layers, the models tend to store gender in a linear fashion.

The last two layers in particular are completely unable to recover the erased gender information. All linear probes trained on these layers show near-random performance, suggesting a complete linear erasure of gender information in these layers. When comparing the results of probes trained on the model with and without linear erasure, it is seen that the erasure operation is most successful in the last layers of the model, i.e., erasure results in a visible, irrecoverable reduction in probe performance. Note that all operations of the model after the last transformer layer are linear so any non-linearly encoded gender present in the last layer’s output is non-consequential.

We conclude that gender can be linearly erased without recovery from the latter layers of Wav2Vec2 and HuBERT⁵.

5.3.2 Effects on downstream ASR

For performing ASR with Wav2Vec2 or HuBERT, representations from the last encoder layer is fed into a linear layer (called the *language modelling head*) which then classifies it into a speech

symbol. Following gender scrubbing, we feed the gender-erased embeddings from the last layer into the language head and measure WER at the output. The language head is a linear layer, so linear erasure at the last layer masks gender from this layer as well. The WER obtained after scrubbing is compared with the original WER obtained when we do not intervene in the transformer’s layers. The comparison is tabulated in [Table 5.2](#) and sheds light on two questions:

1. What are the consequences of erasing gender from the ASR model?
2. Is gender information in the last layers used for ASR despite its ”presence” in these layers?

Dataset	Wav2Vec2		HuBERT	
	Original	Scrubbed	Original	Scrubbed
TIMIT	23.96	24.18	19.23	21.57
Librispeech	2.76	2.77	2.07	2.90

TABLE 5.2: WER before and after gender scrubbing

In most cases, we see that gender scrubbing has a slight albeit noticeable drop on the WER of the model. We note that HuBERT sustains more damage than Wav2Vec2 during linear erasure. This could be because of the fact that the gender probe performances for HuBERT at the last layer are higher than that of the Wav2Vec2 model (Compare the probe performances at the last layer between [Figure 6.4](#) and [Figure 6.3](#)). The small difference in WER means that erasing gender from these layers do not affect the linguistic content, and the model is still able to transcribe the speech segments, i.e., we would have seen a more prominent drop in WER if the eraser was interfering with the linguistic space when removing gender.

We conclude that there is reduced overlap between the linguistic and (linear) gender subspaces in the last layers of Wav2Vec2-ASR and HuBERT-ASR

CHAPTER 6

ON POOLING AND PROBING

In [Figure 5.2](#), we observe that the probe trained without intervention shows comparable performances between layer 10 and layer 21 but the probe trained after linear erasure behaves very differently in these layers. In the analysis section, we ask the following question:

Why is non-recoverable gender removal more successful in the final layers despite comparable original probe performances to the initial layers?

6.0.1 The Problem with Mean Pooling

When training probes at a specific layer of the model, it is common practice to pool the token level representations into a single embedding before training the classifier. The intuition is that the pooling operation acts as a sequence level summarization operation: attributes detected in the pooled embedding has to be encoded at a token level. In this section, we show the problems of this assumption. We start by looking at its converse: whether attributes *absent* in a pooled representation can be present at the token level.

Since the attribute in question is gender and the most common pooling operation in literature is mean pooling, we start by training gender probes on mean pooled representations of Wav2vec2. Using Librispeech as the target dataset, we train mean pooled probes at each layer of the pretrained and ASR finetuned Wav2vec2 models. We also train a weighted pooling operation in the same setup. The weighted pooler uses a learned weight at each position before pooling the representation. The probe performances in each case is shown in [Figure 6.1](#) and the weights learned by the weighter pooler across 10 equidistant positions is shown in [6.2a](#) and [6.2b](#). For both pretrained and fine tuned models, we see that the mean probe outperforms the weighted probe in the early layers, but for both models the weighted probe outperforms the mean probe in the last layers. A closer look at the weight distribution yields two observations:

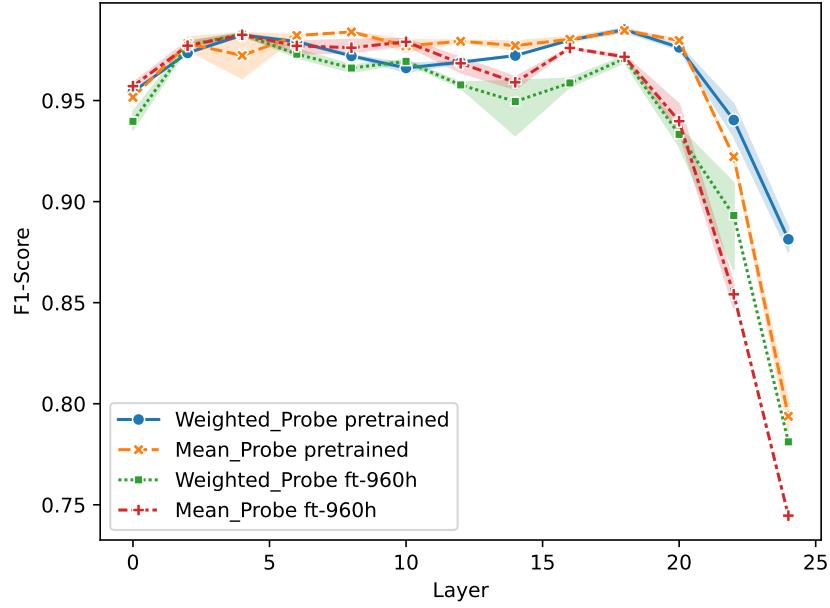


FIGURE 6.1: Pooled probe performances on Librispeech across the hidden layers of pretrained and finetuned Wav2Vec2 models

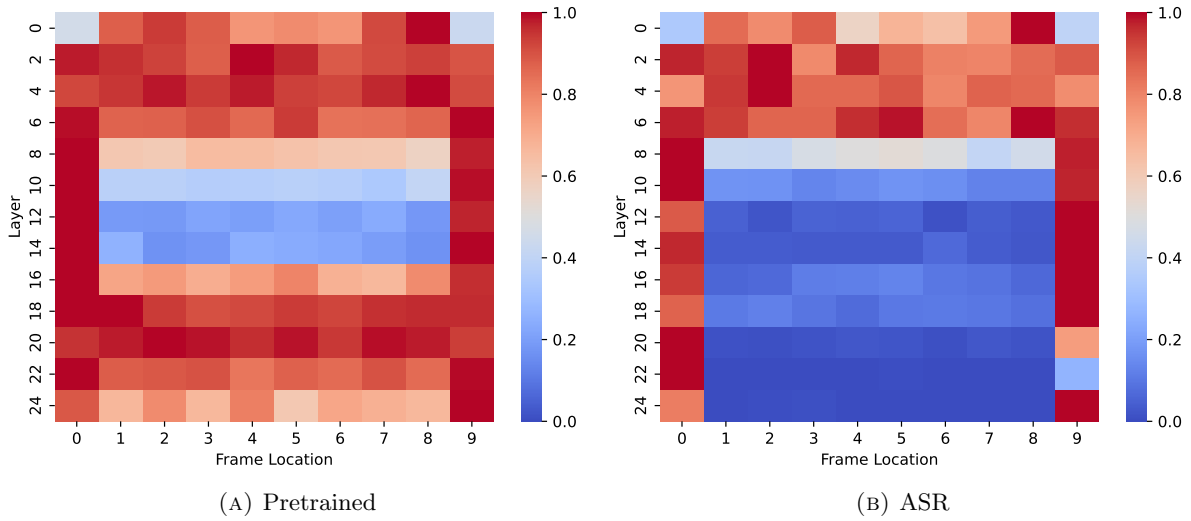


FIGURE 6.2: Distribution of weights for the weighted probe across the layers of Wav2Vec2

1. The weights for the first and last frame positions is considerably and consistently higher than the other positions for both models across many layers
2. For some layers, the weighted probe seems to completely ignore mid-frame representations

Note that at this stage, it is not necessarily clear that gender information is redirected into these positions. We only see that the weighted probe learns to weigh the initial positions heavily and tends to ignore the other positions. This could be a spurious artifact of the probe.

Weighted probing hints that there is a non-uniform frame level organization of gender-information across model layers

6.0.2 Snapshot probing

In the previous section, we trained linear probes on mean pooled layer representations. While the weight distribution of the weighted probe hinted at a heterogeneous organization of gender information at a frame level, it does not concretely tell us if the distribution is an artifact of the weighted probe. Henceforth, we will train probes on a frame-level, meaning we train linear probes on the individual (T, H) representations at each layer. Since this is isolated to the frame position, it gives us a more concrete idea on what is contained in a frame. However, this setup comes with a hurdle: each audio signal $\mathbf{x}_i$ has a different number of time steps and produces an embedding of shape (T_i, H) at each layer. For efficient comparisons, we take 10 equally spaced snapshots across time steps, starting with the first and ending with the last. This forces each layer-wise representation to $(10, H)$ dimensions. We can now train probes on any one of these snapshots, using the same gender label z_i across all snapshots of $\mathbf{x}_i$: we call this **Snapshot probing**. Initially, we train 10 snapshot probes at each layer, training and testing on the *same* snapshot position. The performances of these snapshot probes for the pretrained and fine-tuned models (using Librispeech⁴) is shown in [Figure 6.3](#) and [Figure 6.4](#).

Across the layers of the pretrained models, we see that the probes perform strongly over almost all snapshot positions. Trends in the middle layers resemble the behaviour of an autoencoder, where gender information localizes in the ends. This is consistent with the findings reported in [48], where the authors observe an autoencoder-like tendency within self supervised speech models. Fine-tuning is seen to strongly affect the last layers of the model, where we see an overall reduction in gender-awareness, accompanied by a strong localization of gender into the first and last snapshots (which correspond to the first and last frames of the audio files respectively). In the last four layers of Wav2Vec2 and HuBERT, the performances of the intermediate snapshots fall near-random, suggesting a complete absence of linearly encoded gender. Fine-tuning could be removing gender from the central snapshots in the final layers to make way for linguistic information that facilitates the ASR task. Pretrained HuBERT shows this localization strategy during pretraining already; The final layers additionally *clear up* after fine-tuning.

In [Figure 6.3](#) and [Figure 6.4](#), we also see that the snapshot probes trained at the first and last positions match/outperform the other positions after layer 0 (which is the input to the first transformer layer). This is intriguing, since these positions often contain silence and therefore cannot naturally encode speaker characteristics. The simplest explanation is that the model might be re-purposing these positions to store auxiliary information, utilizing the cross attention mechanism to redirect information into them.

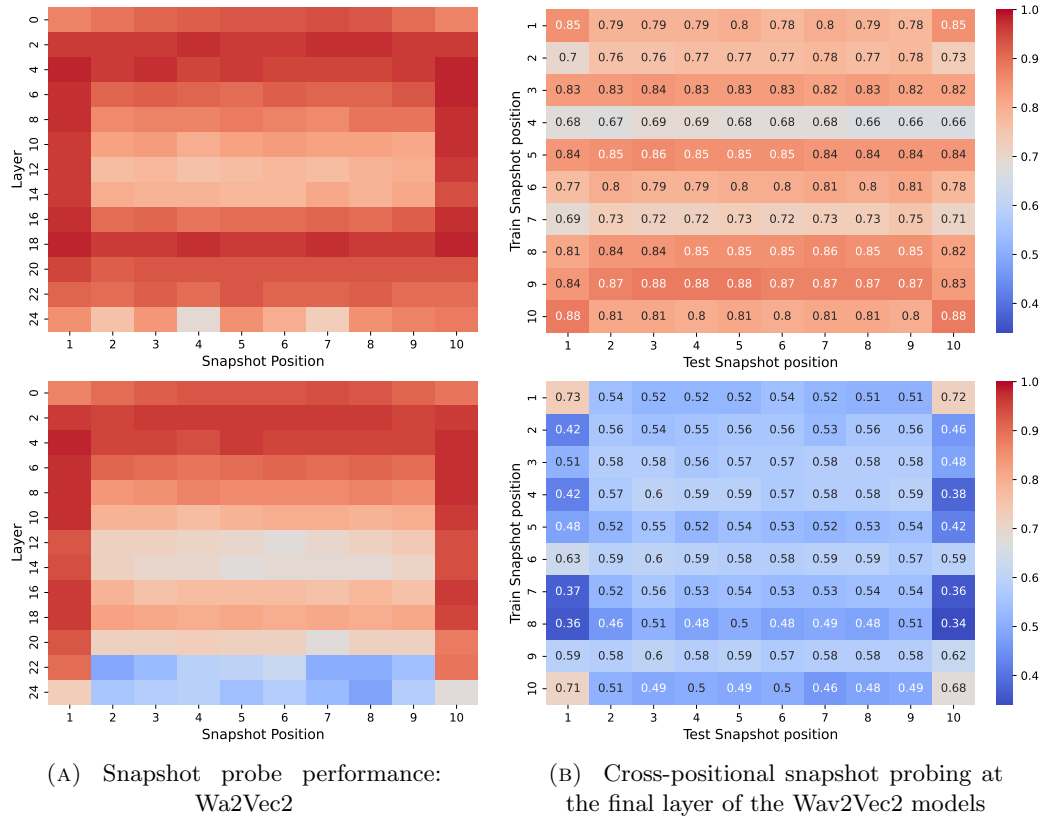


FIGURE 6.3: Snapshot probing on the layers of pretrained (above) and fine-tuned (below) Wav2Vec2. The colors indicate the F1-score of a probed trained at the indicated position. Results are shown for Librispeech.

Frame level probing reveals that the model diverts gender information exclusively into the terminal positions of the ASR model.

6.0.3 Cross-Position Snapshot probing

Figure 6.3 and Figure 6.4 depict snapshot probes trained and tested at the *same* positions in each layer. Alternatively, snapshot probes can be trained and tested on *different* positions within the same (T_i, H) embedding. A high probe performance between two positions then indicates a shared gender-subspace between them. For the Wav2Vec2 model (Similar results are seen for HuBERT in Figure 5.3), results from training and testing snapshot probes at various positions in the final layer is shown in 6.3b. Note that the diagonals of the cross-position plots in 6.3b correspond to the last rows in Figure 6.3. i.e., training and testing on the same probe position. For the pretrained model, we see high transferability across most snapshot positions, which suggests the existence of a shared gender-subspace across frame-level representations in this layer. Trends are different in for the fine-tuned model, where we see a widespread purge of gender from all snapshots except the first and the last. We also see that the probes trained on

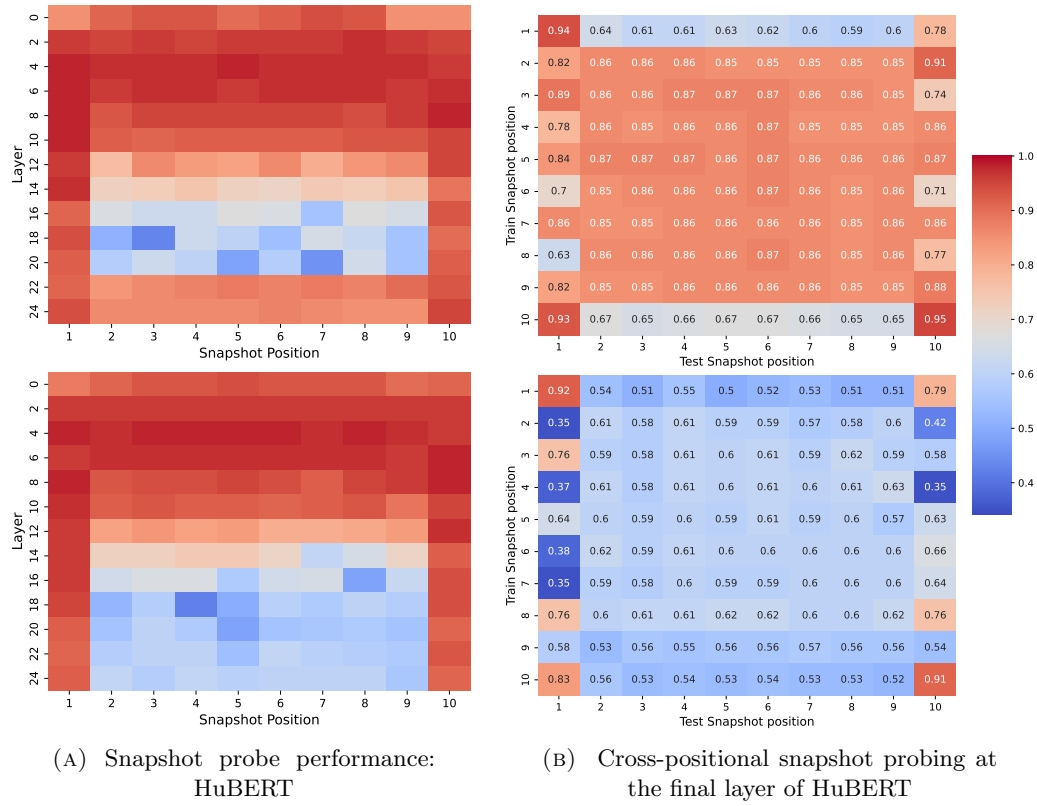


FIGURE 6.4: Snapshot probing on the layers of pretrained (above) and fine-tuned (below) HuBERT. The colors indicate the F1-score of a probed trained at the indicated position. Results are shown for Librispeech.

these positions have a high transferability, which suggests that the gender-subspace is shared only between these positions in the fine tuned model. Clearly, fine tuning re-purposes the gender-subspace in the last layer to store linguistic information in all frames except the first and the last. This adds to our previous conclusions : In the final layer, gender information is stored in the first and last frame positions in a *shared subspace*. The findings in [Figure 6.4](#) and [Figure 6.3](#) help interpret the trends from [Figure 5.2](#): and [Figure 5.3](#)

We can remove gender with reduced recovery from the last layers because it is localized and stored in shared subspaces in these layers, while removal from the initial layers is difficult owing to the omnipresence of gender in these layers.

CHAPTER 7

CONCLUSIONS AND FUTURE WORK

This thesis examined how pretrained transformer models (both linguistic and acoustic) influence the ASR landscape. In the first part, we compared two approaches that approximate GPT-2 into an n-gram model and find that large-scale sampling after fine-tuning performs considerably better than probability-based approximation. We introduced a vocabulary-restricted decoding schema and show that it is helpful in low-resource, domain-specific scenarios. We showed that interpolating the baseline trigram LM from a domain-specific corpus with trigrams made from large-scale generation improves text perplexity by across eight domains. Using dev set perplexity to compare domains, we showed that the sampling volume required to match the baseline trigram perplexity is inversely proportional to the domain modeling difficulty. We also extend text-based augmentation to few-shot scenarios, where we see impressive performance with GPT-2. However, we find that the cost benefit ratio declines rapidly with additional training data and that the trigram model is a tough baseline at higher data sizes.

The second part of this thesis explored the use of linearly detectable gender in two transformer-based ASR models, Wav2Vec2 and HuBERT. We first employ linear erasure to remove gender from each layer of the model and find that it can be removed from the final layers without damaging the ASR performance heavily. Our experiments demonstrate the feasibility of creating gender-neutral ASR embeddings in the linear space. To explain the ease of gender removal from the latter layers, we explore the frame-level organization of gender in the model’s hidden representations. We find that in the latter layers, gender information concentrates in a shared subspace within the embeddings of the initial and final positions. This facilitates the removal of gender cues from the final layers of the model. Our work sheds light on how ASR models encode gender and pave way to creating usable but speaker-blind embeddings.

7.1 Future work

7.1.1 SSL models and feature erasure

Non-Linear erasure Most of our work looked at gender-encoding in the linear domain, but it is clear from our findings that the model encodes gender in non-linear subspaces as well. We did not explore non-linear erasure because such methods did not exist at the time of experimentation. However, subsequent work [71, 72] has demonstrated the potential to erase non-linearly encoded information. Our work can be extended in this direction, to increase the precision of the findings.

Extension to other speech models Our erasure and probing analyses focus on ASR models and their pretrained counterparts. This might not translate across architectures and tasks: For example, Fucci et al. [73] find that the nature and extent of gender encoding in speech translation systems depends on architectural choices. Their results for encoder-decoder models mirror ours but they show that modern architectures that combine speech encoders and translation systems minimally encode gender. Their findings warrant an investigation of other speech systems in this context, like TTS models, Speech LLMs, etc.

Circuit-level understanding This work, and other works investigating gender in speech models [53, 74] study the model’s hidden layer representations to see how gender is encoded and used in these models. This still leaves another question unexplored: “what neurons and circuits drive such phenomena?”. Advances in mechanistic interpretability can help here: techniques like causality tracing [75] or activation patching [76] have been successfully used to find and modify circuits in large language models. Using these to control behavior in speech models can model gender phenomena with greater precision.

7.1.2 Transformers and Speech Models

The field has taken its twists and turns since SSL models paved the pretraining pipeline for speech processing. The following sections are intended to take the reader to the edge of active research and outline open problems in the field.

Transcription and Translation In 2022, OpenAI released Whisper [16], an sequence-to-sequence network that is trained solely with an ASR objective (ie., no pretraining) and 700k hours of audio. Whisper reported state of the art performances for ASR at the time, especially for low resource languages and robust acoustic conditions. The *catch* with whisper was that

(1) It is not an 'all-purpose' encoder like other SSL models so it cannot not be used for non-ASR tasks easily and (2) Whisper generation is auto-regressive, which makes it several orders slower than SSL models. Since the closed-door training of such models hinders good science, researchers are looking into reconstructing them with open resources and academic compute [77, 78]¹. Apart from the open-science narrative, such works enable a better understanding of the training dynamics and the data-related effects that shape model behavior. Training such models in transparent settings remains an active area of research.

Apart from the impressive transcription abilities, Whisper also showed capabilities to translate 99 languages to English (**X-En**) on the fly, which attracted a lot of attention. Practically, this was achieved by changing the decoder prompt, with no additional training or text-based operation whatsoever. i.e., if one changed the decoder prefix from `<|transcribe|><|fr|>` to `<|translate|><|fr|>`, the model continued generation accordingly without having to do anything else. Meta released its Seamless [79] suite of models shortly after, extending translation abilities from X-En to (**X-X**). Unlike Whisper however, seamless is a suite of models that act together in cascade, i.e., an SSL encoder (Wav2Vec2-Bert) $\rightarrow$ machine translation encoder (NLLB [80]) $\rightarrow$ machine translation decoder (NLLB) all adapted to handle each other's inputs and outputs. Notably, this made multilingual X-X translation possible in text and speech with impressive numbers for cross-modal and cross-lingual applications. Naturally, the latency degradation that comes with auto-regressive modeling afflicts this setup as well, making it an undesirable candidate for transcription if translation is not involved. Combining speech transcription with translation remains an active area of research, especially in low-resource scenarios.

Speech LLMs With the development of strong language models like the GPT and LLaMa, research on combining them with SSL models started gaining immense traction. The models produced in this line of research are being called **Speech-Language Models (SLMs)** [81].

Model architecture is an active field of research in the SLM landscape: there is little consensus on what the best approach for LLM+SSL integration is. We discuss three popular SLM architectures here, but the reader is directed to Arora et al. [81] for discussions on more architectures. One of the simplest and popular SLM architectures is SLAM [82], where an LLM is conditioned on the speech embeddings produced by an SSL model. Arora et al. [81] call these *speech-aware-text-LLMs* because they model $p(\text{text} \mid \text{text}, \text{speech})$. The model architecture is illustrated in **Figure 7.1**. Current (September 2025) state of the art SLMs like Audio Flamingo [83] and Qwen-Audio [84] use this architecture to make SLMs that perform well across downstream tasks like question answering, audio description, audio reasoning etc. Another line of research involves the construction of *textless* LLMs by modeling discrete speech units directly (i.e.,

¹[77] won the best paper award at INTERSPEECH 2025 for their work at democratizing whisper-style training, illustrating the current interest and relevance of this kind of work

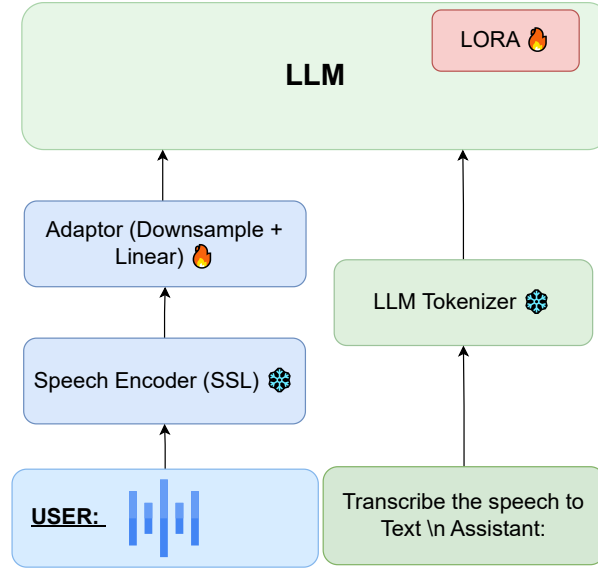


FIGURE 7.1: SLAM architecture for constructing SLMs using an LLM and an SSL encoder. Only a subset of the total parameters are trained (shown with a flame symbol).

$p(\text{speech} \mid \text{speech})$). In this setup, speech is encoded using an SSL model, discretized (using clustering or vector quantization) and then modeled auto-regressively using a transformer model. When trained on large volumes of data, there is some evidence [85, 86] that resulting models can generate coherent speech-completions without the need of written text or text based models. A third architecture jointly models speech and text ($p(\text{speech}, \text{text})$) using a technique called *interleaving*. To train interleaved models, discretized speech sequences and first randomly interleaved with (aligned) text sequences, forming multi-modal sequences. The model is then trained on these sequences using a masked language modeling loss, encouraging strong cross-modal generalization. To this extent, these models have been shown to display ASR and TTS capabilities using a simple few shot prompt [87]. Previously, scaling laws for speech-aware language models were studied with the conclusion that SLMs require 3x more data than text based models to scale [88]. However, recent work has shown that interleaved models scale much better than speech-aware models [89], making them strong contenders for future work.

Interpretability is another critically underexplored theme in the SLM domain: barely any work has looked into the mechanisms that operate on the underlying LLM, forcing it to adapt, transfer abilities or interact with the new modality. Future research should look into the mechanisms that drive these architectural choices rather than benchmark performances.

LIST OF FIGURES

Figure 1.1	Block diagram of an ASR model	2
Figure 2.1	Illustrating the manipulation of the output logits for Vocabulary restriction. Probabilities for unwanted BPE tokens are made to vanish manually and the distribution is re-normalized to predict from the desired subset.	7
Figure 2.2	Illustrating the differences between Vocabulary restriction (right) and Lexical restriction (left) on a sample vocabulary space	8
Figure 2.3	Interpolation with sampled n-gram models reduce baseline perplexity across all datasets. All perplexities normalized against the respective KN3	11
Figure 3.1	Benefits of sampling depends on domain difficulty. Perplexities are normalized across the respective KN3, which is shown as the horizontal line at 1.0. Darker colour denotes higher perplexity on the dev-set	13
Figure 3.2	Performance of language models across sub-sampled training corpora. Perplexities are normalized with the full-scale KN3 perplexity (which is shown as the blue dotted line at 1.0). The Taskmaster datasets are averaged to generate one plot.	13
Figure 4.1	SSL pretraining architecture	17
Figure 5.1	Steps used for LEACE. Figure sourced from Belrose et al. [66]	21
Figure 5.2	Gender scrubbing for the Wav2Vec2 ASR model. Plots depict linear probe performances at the input and output when linearly erasing gender from the input at each layer. Mean probe performance on the original model is also shown. . . .	23
Figure 5.3	Gender scrubbing for the HuBERT ASR model. Plots depict linear probe performances at the input and output when linearly erasing gender from the input at each layer. Mean probe performance on the original model is also shown. . . .	24
Figure 6.1	Pooled probe performances on Librispeech across the hidden layers of pretrained and finetuned Wav2Vec2 models	27
Figure 6.2	Distribution of weights for the weighted probe across the layers of Wav2Vec2	27

Figure 6.3	Snapshot probing on the layers of pretrained (above) and fine-tuned (below) Wav2Vec2. The colors indicate the F1-score of a probed trained at the indicated position. Results are shown for Librispeech.	29
Figure 6.4	Snapshot probing on the layers of pretrained (above) and fine-tuned (below) HuBERT. The colors indicate the F1-score of a probed trained at the indicated position. Results are shown for Librispeech.	30
Figure 7.1	SLAM architecture for constructing SLMs using an LLM and an SSL encoder. Only a subset of the total parameters are trained (shown with a flame symbol).	34

LIST OF TABLES

Table 2.1	Dataset distribution and splits.	6
Table 2.2	Sampling-based approximation of GPT-2 into an n-gram model consistently outperforms probability-based approximation	9
Table 2.3	Perplexities and OOV rates for the n-gram models (Movies, Restaurant, Music, Flights). Metrics are <i>Perplexity/#OOVs</i>	10
Table 2.4	Perplexities and OOV rates for the n-gram models (Sports, Food, ATCO2, HUB4). Metrics are <i>Perplexity/#OOVs</i>	10
Table 5.1	Dataset statistics for TIMIT and Librispeech. Note that there is no speaker overlap between the train and test sets	21
Table 5.2	WER before and after gender scrubbing	25

BIBLIOGRAPHY

- [1] Mehryar Mohri, Fernando Pereira, and Michael Riley. Weighted finite-state transducers in speech recognition. *Computer Speech & Language*, 16(1):69–88, 2002.
- [2] Frederick Jelinek. *Statistical methods for speech recognition*. MIT press, 1998.
- [3] Steve Young, Gunnar Evermann, Mark Gales, Thomas Hain, Dan Kershaw, Xunying Liu, Gareth Moore, Julian Odell, Dave Ollason, Dan Povey, et al. The HTK book. *Cambridge university engineering department*, 3(175):12, 2002.
- [4] L Bahl, Raimo Bakis, P Cohen, A Cole, Frederick Jelinek, B Lewis, and R Mercer. Further results on the recognition of a continuously read natural corpus. In *ICASSP’80. IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 5, pages 872–875. IEEE, 1980.
- [5] Stanley F Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–394, 1999.
- [6] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning*, pages 369–376, 2006.
- [7] Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal processing magazine*, 29(6):82–97, 2012.
- [8] Dimitri Palaz, Mathew Magimai-Doss, Ronan Collobert, et al. Analysis of CNN-based speech recognition system using raw speech as input. In *Interspeech*, pages 11–15. Dresden, Germany, 2015.
- [9] Jui-Ting Huang, Jinyu Li, and Yifan Gong. An analysis of convolutional neural networks for speech recognition. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4989–4993. IEEE, 2015.
- [10] Apeksha Shewalkar. Performance evaluation of deep neural networks applied to speech recognition: RNN, LSTM and GRU. *Journal of Artificial Intelligence and Soft Computing Research*, 9, 2019.

- [11] Alex Graves, Navdeep Jaitly, and Abdel-rahman Mohamed. Hybrid speech recognition with deep bidirectional LSTM. In *2013 IEEE workshop on automatic speech recognition and understanding*, pages 273–278. IEEE, 2013.
- [12] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. *CoRR*, abs/1409.0473, 2014. URL <https://api.semanticscholar.org/CorpusID:11212020>.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [14] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations, 2020. URL <https://arxiv.org/abs/2006.11477>.
- [15] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.
- [16] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning*, pages 28492–28518. PMLR, 2023.
- [17] Yen-Lu Chow and Richard Schwartz. The n-best algorithm: Efficient procedure for finding top n sentence hypotheses. In *Speech and Natural Language: Proceedings of a Workshop Held at Cape Cod, Massachusetts, October 15-18, 1989*, 1989.
- [18] Saket Dingliwal, Ashish Shenoy, Sravan Bodapati, Ankur Gandhe, Ravi Teja Gadde, and Katrin Kirchhoff. Domain prompts: Towards memory and compute efficient domain adaptation of ASR systems. *arXiv preprint arXiv:2112.08718*, 2021.
- [19] Takuma Udagawa, Masayuki Suzuki, Gakuto Kurata, Nobuyasu Itoh, and George Saon. Effect and analysis of large-scale language model rescoring on competitive ASR systems. *arXiv preprint arXiv:2204.00212*, 2022.
- [20] Xianrui Zheng, Chao Zhang, and Philip C Woodland. Adapting GPT, GPT-2 and BERT language models for speech recognition. In *2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 162–168. IEEE, 2021.

- [21] Anoop Deoras, Tomáš Mikolov, Stefan Kombrink, Martin Karafiát, and Sanjeev Khudanpur. Variational approximation of long-span language models for lvc sr. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5532–5535, 2011. doi: 10.1109/ICASSP.2011.5947612.
- [22] Holger Schwenk and Jean-Luc Gauvain. Connectionist language modeling for large vocabulary continuous speech recognition. In *2002 IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 1, pages I-765–I-768, 2002. doi: 10.1109/ICASSP.2002.5743830.
- [23] Mittul Singh, Youssef Oualil, and Dietrich Klakow. Approximated and Domain-Adapted LSTM Language Models for First-Pass Decoding in Speech Recognition. In *INTER-SPEECH*, pages 2720–2724, 2017.
- [24] Balázs Tarján, György Szaszák, Tibor Fegyó, and Péter Mihajlik. On the effectiveness of neural text generation based data augmentation for recognition of morphologically rich speech. In *Text, Speech, and Dialogue: 23rd International Conference, TSD 2020, Brno, Czech Republic, September 8–11, 2020, Proceedings 23*, pages 437–445. Springer, 2020.
- [25] Ke Li, Hainan Xu, Yiming Wang, Daniel Povey, and Sanjeev Khudanpur. Recurrent neural network language model adaptation for conversational speech recognition. In *Interspeech*, volume 2018, pages 3373–3377, 2018.
- [26] Heike Adel, Katrin Kirchhoff, Ngoc Thang Vu, Dominic Telaar, and Tanja Schultz. Comparing approaches to convert recurrent neural networks into backoff language models for efficient decoding. In *Fifteenth Annual Conference of the International Speech Communication Association*, 2014.
- [27] Yiren Wang, Hongzhao Huang, Zhe Liu, Yutong Pang, Yongqiang Wang, ChengXiang Zhai, and Fuchun Peng. Improving N-gram Language Models with Pre-trained Deep Transformer. *CoRR*, abs/1911.10235, 2019.
- [28] Alec Radford, Jeff Wu, Rewon Child, D. Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. 2019.
- [29] Bill Byrne, Karthik Krishnamoorthi, Chinnadhurai Sankar, Arvind Neelakantan, Daniel Duckworth, Semih Yavuz, Ben Goodrich, Amit Dubey, Andy Cedilnik, and Kyu-Young Kim. Taskmaster-1: Toward a realistic and diverse dialog dataset. *arXiv preprint arXiv:1909.05358*, 2019.
- [30] David Graff, Zhibiao Wu, Robert MacIntyre, and Mark Liberman. The 1996 broadcast news speech and language-model corpus. In *Proceedings of the DARPA Workshop on Spoken Language technology*, pages 11–14. Citeseer, 1997.

- [31] Juan Zuluaga-Gomez, Karel Veselý, Igor Szöke, Petr Motlicek, Martin Kocour, Mickael Rigault, Khalid Choukri, Amrutha Prasad, Seyyed Saeed Sarfjoo, Iuliia Nigmatulina, et al. ATCO2 corpus: A Large-Scale Dataset for Research on Automatic Speech Recognition and Natural Language Understanding of Air Traffic Control Communications. *arXiv preprint arXiv:2211.04054*, 2022.
- [32] Ebru Arisoy, Stanley F. Chen, Bhuvana Ramabhadran, and Abhinav Sethy. Converting Neural Network Language Models into back-off language models for efficient decoding in automatic speech recognition. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 8242–8246, 2013. doi: 10.1109/ICASSP.2013.6639272.
- [33] Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. *arXiv preprint arXiv:1704.07138*, 2017.
- [34] J Edward Hu, Huda Khayrallah, Ryan Culkin, Patrick Xia, Tongfei Chen, Matt Post, and Benjamin Van Durme. Improved lexically constrained decoding for translation and monolingual rewriting. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 839–850, 2019.
- [35] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL <https://aclanthology.org/2020.emnlp-demos.6>.
- [36] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.
- [37] Andreas Stolcke. SRILM-an extensible language modeling toolkit. In *Seventh international conference on spoken language processing*, 2002.
- [38] Sabrina J. Mielke. Can you compare perplexity across different segmentations?, Apr 2019. URL <https://sjmielke.com/comparing-perplexities.htm>.
- [39] Ryan Cotterell, Sabrina J. Mielke, Jason Eisner, and Brian Roark. Are All Languages Equally Hard to Language-Model? In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 536–541, New Orleans, Louisiana, June

2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-2085. URL <https://aclanthology.org/N18-2085>.
- [40] Shu-Wen Yang, Po-Han Chi, Yung-Sung Chuang, Cheng-I Lai, Kushal Lakhotia, Yist Y. Lin, Andy T. Liu, Jiatong Shi, Xuankai Chang, Guan-Ting Lin, Tzu hsien Huang, Wei-Cheng Tseng, Ko tik Lee, Da-Rong Liu, Zili Huang, Shuyan Dong, Shang-Wen Li, Shinji Watanabe, Abdel rahman Mohamed, and Hung yi Lee. SUPERB: Speech processing Universal PERformance Benchmark. In *Interspeech*, 2021. URL <https://api.semanticscholar.org/CorpusID:233481577>.
- [41] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in neural information processing systems*, 33:12449–12460, 2020.
- [42] Eric Jang, Shixiang Gu, and Ben Poole. Categorical Reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=rkE3y85ee>.
- [43] Chung-Cheng Chiu, James Qin, Yu Zhang, Jiahui Yu, and Yonghui Wu. Self-supervised Learning with Random-projection Quantizer for Speech Recognition. In *International Conference on Machine Learning*, 2022. URL <https://api.semanticscholar.org/CorpusID:246608081>.
- [44] Ryan Whetten, Titouan Parcollet, Marco Dinarelli, and Yannick Estève. Open Implementation and Study of Best-RQ for Speech Processing. *2024 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*, pages 460–464, 2024. URL <https://api.semanticscholar.org/CorpusID:269614311>.
- [45] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, et al. Wavlm: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.
- [46] Abdel-Rahman Mohamed, Hung yi Lee, Lasse Borgholt, Jakob Drachmann Havtorn, Joakim Edin, C. Igel, Katrin Kirchhoff, Shang-Wen Li, Karen Livescu, Lars Maaløe, Tara N. Sainath, and Shinji Watanabe. Self-Supervised Speech Representation Learning: A Review. *IEEE Journal of Selected Topics in Signal Processing*, 16:1179–1210, 2022. URL <https://api.semanticscholar.org/CorpusID:248987289>.
- [47] Ankita Pasad, Bowen Shi, and Karen Livescu. Comparative Layer-Wise Analysis of Self-Supervised Speech Models. *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2022. URL <https://api.semanticscholar.org/CorpusID:253397618>.

- [48] Ankita Pasad, Ju-Chieh Chou, and Karen Livescu. Layer-wise analysis of a self-supervised speech representation model. In *2021 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 914–921. IEEE, 2021.
- [49] Ankita Pasad, Chung-Ming Chien, Shane Settle, and Karen Livescu. What Do Self-Supervised Speech Models Know About Words? *Transactions of the Association for Computational Linguistics*, 12:372–391, 2023. URL <https://api.semanticscholar.org/CorpusID:259316239>.
- [50] Kwanghee Choi, Ankita Pasad, Tomohiko Nakamura, Satoru Fukayama, Karen Livescu, and Shinji Watanabe. Self-Supervised Speech Representations are More Phonetic than Semantic. In *Interspeech 2024*, pages 4578–4582, 2024. doi: 10.21437/Interspeech.2024-1157.
- [51] Maureen de Seyssel and Marvin Lavechin and Yossi Adi and Emmanuel Dupoux and Guillaume Wisniewski. Probing phoneme, language and speaker information in unsupervised speech representations. In *Interspeech 2022*, pages 1402–1406, 2022. doi: {10.21437/Interspeech.2022-373}.
- [52] Chiyu Feng, Po chun Hsu, and Hung yi Lee. Silence is Sweeter Than Speech: Self-Supervised Model Using Silence to Store Speaker Information. *ArXiv*, abs/2205.03759, 2022. URL <https://api.semanticscholar.org/CorpusID:248571940>.
- [53] Giuseppe Attanasio, Beatrice Savoldi, Dennis Fucci, and Dirk Hovy. Multilingual Speech Models for Automatic Speech Recognition Exhibit Gender Performance Gaps. *arXiv preprint arXiv:2402.17954*, 2024.
- [54] Siyuan Feng, Bence Mark Halpern, Olya Kudina, and Odette Scharenborg. Towards inclusive automatic speech recognition. *Computer Speech & Language*, 84:101567, 2024.
- [55] Mahault Garnerin, Solange Rossato, and Laurent Besacier. Gender representation in French broadcast corpora and its impact on ASR performance. In *Proceedings of the 1st international workshop on AI for smart TV content production, access and delivery*, pages 3–9, 2019.
- [56] Marcio Fuckner, Sophie Horsman, Pascal Wiggers, and Iskaj Janssen. Uncovering Bias in ASR Systems: Evaluating Wav2vec2 and Whisper for Dutch speakers. In *2023 International Conference on Speech Technology and Human-Computer Dialogue (SpeD)*, pages 146–151. IEEE, 2023.
- [57] Martine Adda-Decker and Lori Lamel. Do speech recognizers prefer female speakers? In *Ninth European Conference on Speech Communication and Technology*, 2005.
- [58] Rachael Tatman. Gender and dialect bias in YouTube’s automatic captions. In *Proceedings of the first ACL workshop on ethics in natural language processing*, pages 53–59, 2017.

- [59] Eiman Alsharhan and Allan Ramsay. Investigating the effects of gender, dialect, and training size on the performance of Arabic speech recognition. *Language Resources and Evaluation*, 54:975–998, 2020.
- [60] Beatrice Savoldi, Marco Gaido, Luisa Bentivogli, Matteo Negri, and Marco Turchi. Under the morphosyntactic lens: A multifaceted evaluation of gender bias in speech translation. *arXiv preprint arXiv:2203.09866*, 2022.
- [61] Marta R Costa-Jussà, Christine Basta, and Gerard I Gállego. Evaluating gender bias in speech translation. *arXiv preprint arXiv:2010.14465*, 2020.
- [62] Cristina Gorrostieta, Reza Lotfian, Kye Taylor, Richard Brutti, and John Kane. Gender De-Biasing in Speech Emotion Recognition. In *Interspeech*, pages 2823–2827, 2019.
- [63] Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics*, 9:160–175, 2021.
- [64] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [65] Shauli Ravfogel, Yanai Elazar, Hila Gonen, Michael Twiton, and Yoav Goldberg. Null it out: Guarding protected attributes by iterative nullspace projection. *arXiv preprint arXiv:2004.07667*, 2020.
- [66] Nora Belrose, David Schneider-Joseph, Shauli Ravfogel, Ryan Cotterell, Edward Raff, and Stella Biderman. Leace: Perfect linear concept erasure in closed form. *Advances in Neural Information Processing Systems*, 36, 2024.
- [67] Shauli Ravfogel, Michael Twiton, Yoav Goldberg, and Ryan D Cotterell. Linear adversarial concept erasure. In *International Conference on Machine Learning*, pages 18400–18421. PMLR, 2022.
- [68] John S Garofolo, Lori F Lamel, William M Fisher, Jonathan G Fiscus, and David S Pallett. DARPA TIMIT acoustic-phonetic continuous speech corpus CD-ROM. NIST speech disc 1-1.1. *NASA STI/Recon technical report n*, 93:27403, 1993.
- [69] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: an ASR corpus based on public domain audio books. In *Acoustics, Speech and Signal Processing (ICASSP), 2015 IEEE International Conference on*, pages 5206–5210. IEEE, 2015.

- [70] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [71] Lucia Quirke and Nora Belrose. Slowing Learning by Erasing Simple Features, 2025. URL <https://arxiv.org/abs/2502.02820>.
- [72] Antoine Saillenfest and Pirmin Lemberger. Nonlinear Concept Erasure: a Density Matching Approach. *arXiv preprint arXiv:2507.12341*, 2025.
- [73] Dennis Fucci, Marco Gaido, Matteo Negri, Luisa Bentivogli, Andre Martins, and Giuseppe Attanasio. Different Speech Translation Models Encode and Translate Speaker Gender Differently. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1005–1019, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-252-7. doi: 10.18653/v1/2025.acl-short.78. URL <https://aclanthology.org/2025.acl-short.78/>.
- [74] Severine Guillaume, Maxime Fily, Alexis Michaud, and Guillaume Wisniewski. Gender and Language Identification in Multilingual Models of Speech: Exploring the Genericity and Robustness of Speech Representations. *Interspeech 2024*, 2024. URL <https://api.semanticscholar.org/CorpusID:272345035>.
- [75] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022.
- [76] Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- [77] Yifan Peng and Muhammad Shakeel and Yui Sudo and William Chen and Jinchuan Tian and Chyi-Jiunn Lin and Shinji Watanabe. OWSM v4: Improving Open Whisper-Style Speech Models via Data Scaling and Cleaning. In *Interspeech 2025*, pages 2225–2229, 2025. doi: {10.21437/Interspeech.2025-1062}.
- [78] Krishna C. Puvvada, Piotr Żelasko, He Huang, Oleksii Hrinchuk, Nithin Rao Koluguri, Kunal Dhawan, Somshubra Majumdar, Elena Rastorgueva, Zhehuai Chen, Vitaly Lavrukhin, Jagadeesh Balam, and Boris Ginsburg. Less is More: Accurate Speech Recognition Translation without Web-Scale Data. In *Interspeech 2024*, pages 3964–3968, 2024. doi: 10.21437/Interspeech.2024-2294.

- [79] Loïc Barrault, Yu-An Chung, Mariano Coria Meglioli, David Dale, Ning Dong, Mark Dupont, Paul-Ambroise Duquenne, Brian Ellis, Hady Elsahar, Justin Haaheim, et al. Seamless: Multilingual Expressive and Streaming Speech Translation. *arXiv preprint arXiv:2312.05187*, 2023.
- [80] Marta R Costa-Jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672*, 2022.
- [81] Siddhant Arora, Kai-Wei Chang, Chung-Ming Chien, Yifan Peng, Haibin Wu, Yossi Adi, Emmanuel Dupoux, Hung-Yi Lee, Karen Livescu, and Shinji Watanabe. On The Landscape of Spoken Language Models: A Comprehensive Survey, 2025. URL <https://arxiv.org/abs/2504.08528>.
- [82] Ziyang Ma, Guanrou Yang, Yifan Yang, Zhifu Gao, Jiaming Wang, Zhihao Du, Fan Yu, Qian Chen, Siqi Zheng, Shiliang Zhang, and Xie Chen. An Embarrassingly Simple Approach for LLM with Strong ASR Capacity. *CoRR*, abs/2402.08846, 2024. URL <https://doi.org/10.48550/arXiv.2402.08846>.
- [83] Arushi Goel, Sreyan Ghosh, Jaehyeon Kim, Sonal Kumar, Zhifeng Kong, Sang gil Lee, Chao-Han Huck Yang, Ramani Duraiswami, Dinesh Manocha, Rafael Valle, and Bryan Catanzaro. Audio Flamingo 3: Advancing Audio Intelligence with Fully Open Large Audio Language Models, 2025. URL <https://arxiv.org/abs/2507.08128>.
- [84] Yunfei Chu, Jin Xu, Xiaohuan Zhou, Qian Yang, Shiliang Zhang, Zhijie Yan, Chang Zhou, and Jingren Zhou. Qwen-Audio: Advancing Universal Audio Understanding via Unified Large-Scale Audio-Language Models. *ArXiv*, abs/2311.07919, 2023. URL <https://api.semanticscholar.org/CorpusID:265157993>.
- [85] Zalán Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, Matt Sharifi, Dominik Roblek, Olivier Teboul, David Grangier, Marco Tagliasacchi, et al. Audioldm: a language modeling approach to audio generation. *IEEE/ACM transactions on audio, speech, and language processing*, 31:2523–2533, 2023.
- [86] Kushal Lakhotia, Eugene Kharitonov, Wei-Ning Hsu, Yossi Adi, Adam Polyak, Benjamin Bolte, Tu-Anh Nguyen, Jade Copet, Alexei Baevski, Abdelrahman Mohamed, and Emmanuel Dupoux. On generative spoken language modeling from raw audio. *Transactions of the Association for Computational Linguistics*, 9:1336–1354, 2021. doi: 10.1162/tacl_a-00430. URL <https://aclanthology.org/2021.tacl-1.79>.
- [87] Tu Anh Nguyen, Benjamin Muller, Bokai Yu, Marta R Costa-Jussa, Maha Elbayad, Sravya Popuri, Christophe Ropers, Paul-Ambroise Duquenne, Robin Algayres, Ruslan Mavlyutov,

- et al. Spirit-lm: Interleaved spoken and written language model. *Transactions of the Association for Computational Linguistics*, 13:30–52, 2025.
- [88] Santiago Cuervo and Ricard Marxer. Scaling properties of speech language models. *arXiv preprint arXiv:2404.00685*, 2024.
- [89] Gallil Maimon, Michael Hassid, Amit Roth, and Yossi Adi. Scaling analysis of interleaved speech-text language models. *arXiv preprint arXiv:2504.02398*, 2025.